BUILD, TEST AND FLY

Quadcopter Build Guide ESP32-S3 Flight Controller

From bare motors to a drone that holds its height and lands by itself: wiring, ESC calibration, motor adjustment, receiver binding, sensor setup, the flight code, PID tuning, altitude hold, automatic landing, battery management and flight logs.

Frame	F330, XT60 PDB with low-ESR capacitor
Motors / ESCs	4 × A2212 2200KV, 4 × 20–30 A ESC, 6-inch props (6045)
Battery	3S LiPo, 3300 mAh
Controller	Your PCB + ESP32-S3 DevKit (N16R8) + MPU6050 (GY-521) + BMP280
Radio	FlySky FS-i6X transmitter + FS-iA10B receiver (i-BUS)
Project	folder `esp32-drone-fc` (ready for GitHub): 4 sketches + bench test, Python log tool, simulator, documentation. Sketch listings in the appendix.

Version 3 · 15 October 2026 (flight controller v3: altitude hold, automatic landing, battery management, buzzer and LED, flight log; new failsafe settings in Step 7.6) · Read Part 1 completely before plugging in a battery.

Contents

Part 1 — Before you start

1.1 What's in the project folder
1.2 Tools and spare parts
1.3 Safety rules
1.4 How everything connects
1.5 Arduino IDE setup (Step 0)

Part 2 — Motors and ESCs

Step 1 — Wire the ESC signal leads to H6
Step 2 — Arm and calibrate the ESCs (beeps, wiring check)
Step 3 — Test each motor alone
Step 4 — All motors together, emergency stop, signal speed
Step 5 — Motor checks and adjustment
Step 6 — Props on: balance and tethered thrust test

Part 3 — Radio and sensor

Step 7 — Receiver: connect, bind, set up, failsafe
Step 8 — MPU6050: mount, orientation, level calibration
Step 9 — Power for flight + final wiring check

Part 4 — Flying

Step 10 — The flight controller code and bench test
Step 11 — First hover
Step 12 — PID tuning

Part 5 — Version 3: smarter and safer

Step 13 — Version 3 setup: files, buzzer, LED, battery
Step 14 — ALT HOLD: the drone holds its height
Step 15 — LAND and automatic landings
Step 16 — Flight log: see what the drone did
Step 17 — Put the project on GitHub

Part 6 — Reference

LiPo battery care
Pre-flight checklist (print this)
Troubleshooting
Pin map and radio channels
Glossary
Resources: books, papers, courses, documentation

Appendix — Source code

A. 1_motor_test.ino
B. 2_receiver_test.ino

C. 3_imu_test.ino

D. 4_flight_controller (12 files)

Part 1 — Before you start

Every step in this guide tests one thing. If something fails, you know exactly where to look. Do the steps in order: each one depends on the one before.

1.1 What's in the project folder

Sketch	Used in	What it does
`1_motor_test`	Steps 2–6	ESC calibration (timed for you), spin one motor or all, emergency stop (key, BOOT button, auto-stop after 3 s), beep decoder, wiring check, signal-speed test
`2_receiver_test`	Steps 7, 14	Shows every radio channel live, ARM / SAFE, MANUAL / ALT HOLD / LAND, signal lost/OK, failsafe check
`3_imu_test`	Step 8	Checks the MPU6050, finds how it's mounted, measures the level offsets
`4_flight_controller`	Steps 10–16	The flight code: self-levelling, ALT HOLD, LAND, failsafes, battery management, buzzer/LED, flight log. Settings in `config.h`
`extras/bench_test`	any time	All-in-one bench check (props off)
`tools/`	Steps 16, 17	`flight_log.py` (download and plot logs) and the simulator

The sketches are in `esp32-drone-fc/firmware/`. Open each `.ino` from its own folder: Arduino needs the folder and the file to have the same name. `README.md` and `docs/` (ten chapters) explain the code in depth; this guide is the hands-on path.

1.2 Tools and spare parts

Tools

- Soldering iron, solder, flux
- **Multimeter** with continuity (beep) mode
- Small screwdrivers / hex keys for the motor screws
- Heatshrink tube, zip ties, electrical tape
- USB-C **data** cable (some only charge)
- Laptop with Arduino IDE 2.x

Battery and spares

- **Balance charger** for 3S LiPo
- LiPo-safe bag or metal box for charging
- LiPo voltage alarm (cheap beeper on the balance plug)
- Spare props: at least 2 CW + 2 CCW
- A heavy board or box to strap the drone to (Step 6)
- Optional: 5 V 3 A UBEC (Step 9)

1.3 Safety rules

READ THESE TWICE

1. **Props OFF** for every test until Step 6 tells you to put them on. A 6-inch prop at full speed cuts skin deeply.
2. **The battery is the real off switch.** Keep the XT60 connector within reach. Pulling it apart stops everything, even if the code freezes.
3. Plug the LiPo in **last** and unplug it **first**.
4. Never stand in line with the props. Stand behind the drone; keep your face out of the prop plane.
5. Never charge a LiPo unattended or on something that burns. Charge in a LiPo bag.
6. Stop flying at **3.5 V per cell** (10.5 V for 3S). Never go below 3.0 V per cell.
7. First flights outside, in an open field, nobody within 15 m, little wind.

1.4 How everything connects

Thick battery power only runs LiPo → PDB → ESCs → motors. Everything that touches your PCB is a thin signal wire or a 5 V supply.

Power flows from the LiPo; the PCB only sends signals

Flight wiring. For bench tests: H1 unplugged, board on USB.

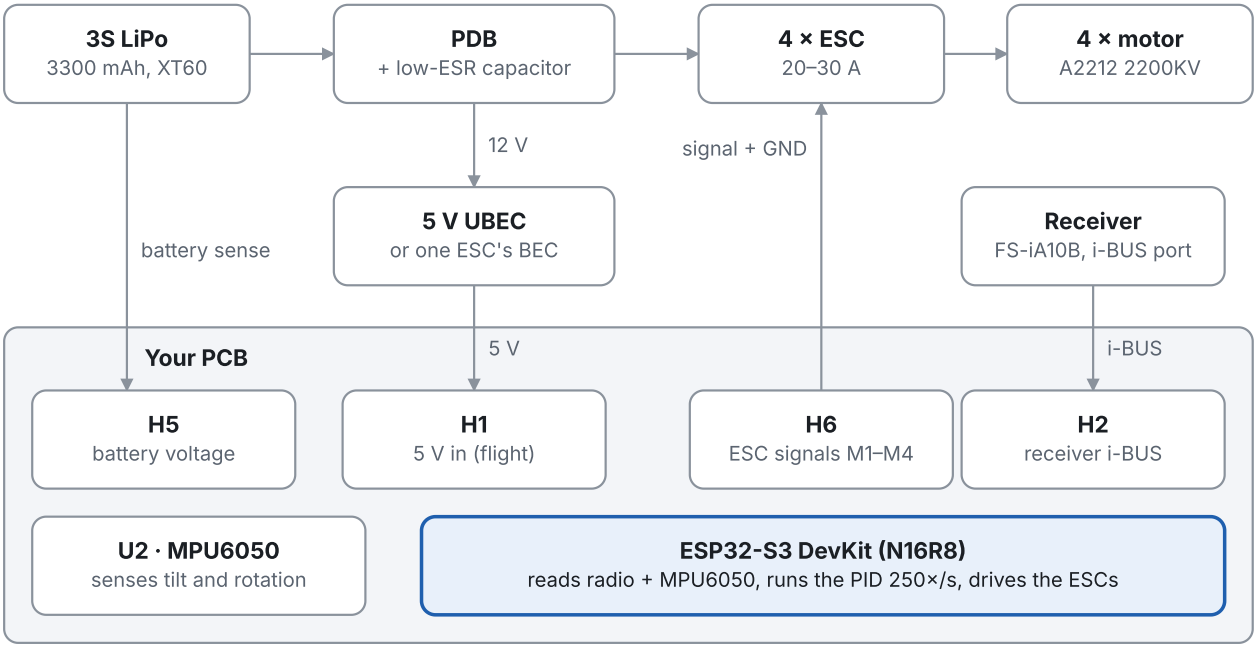


Figure 1 — Flight wiring. The ESP32 never carries motor current.

1.5 Arduino IDE setup (Step 0)

Do this once with the new DevKit **off the PCB**, on a USB cable only. If the first upload works on its own, any later problem is the PCB, not the board.

1. **File → Preferences → Additional boards manager URLs**: add `https://espressif.github.io/arduino-esp32/package_esp32_index.json`
2. **Tools → Board → Boards Manager**: search *esp32* (by Espressif Systems), install the latest version.
3. Plug the cable into the USB-C port marked **COM** (or UART). If Windows shows no new COM port, install the **CH343** or **CP210x** driver (the chip name is printed near that port).
4. Set the Tools menu:

Setting	Value	Why
Board	ESP32S3 Dev Module	your chip
USB CDC On Boot	**Disabled**	Serial goes through the COM port; printing never freezes the code when USB is unplugged
Flash Size	16MB (128Mb)	N16 = 16 MB flash
PSRAM	**OPI PSRAM**	R8 = 8 MB octal PSRAM; wrong value = crash at boot
Partition Scheme	**16M Flash (3MB APP/9.9MB FATFS)**	uses the full flash; the flight logs are saved in the FATFS part
Flash Mode	QIO 80MHz	
Upload Speed	115200	raise to 921600 once uploads work

5. Upload `1_motor_test.ino`. If it stops at `Connecting....`: **hold BOOT, tap RST, release BOOT** while it is still trying.
6. Open the **Serial Monitor** at **115200** baud, line ending **Newline**. Press RST: the motor test menu prints.
7. Put the DevKit on the PCB and press RST again. The menu must still print. If not, something on the PCB pulls a pin: find it before going on.

> **UPLOAD STILL FAILS?**
>
> Try the other USB-C port (marked USB) with *USB CDC On Boot = Enabled*: unplug, hold BOOT, plug in, release BOOT, choose the new COM port, upload, then press RST. Also make sure nothing is plugged into H4 (it shares the upload pins GPIO 43/44).

Part 2 — Motors and ESCs

Here you prove that each ESC hears the board, each motor is in the right place and spins the right way, and that you can stop everything instantly. Props stay off until Step 6.

Step 1 Wire the ESC signal leads to H6

Each ESC has a thin 3-wire servo lead. Only **two** of its wires go to H6: the **signal** (tells the ESC how fast to spin) and the **ground** (gives that signal a common reference).

Motor layout seen from above: diagonal motors spin the same way

Quad X, "props in". The flight code depends on this exact numbering.

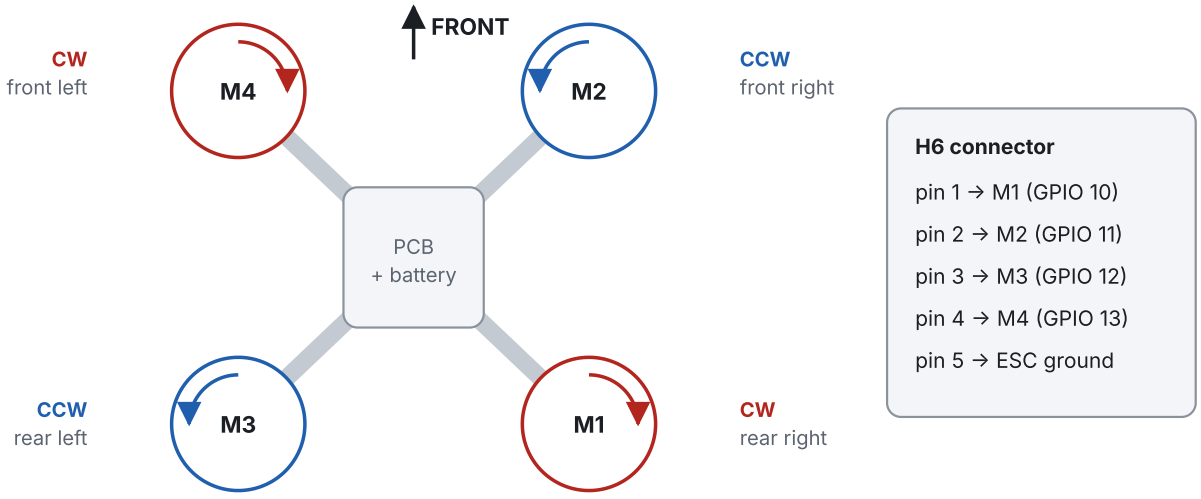


Figure 2 — Motor numbers, positions and spin directions (red = clockwise, blue = counter-clockwise).

How to connect each servo lead

- **White or yellow** (signal) → that motor's H6 pin (1 to 4).
- **Black or brown** (ground) → H6 pin 5. One ESC ground is enough because the grounds are already joined through the PDB; connecting all four is also fine.
- **Red** (5 V from the ESC's BEC) → **not connected** for now. Pull the pin out of the plastic housing or cover it with tape. Step 9 uses one of them.

Mark the **front** of the frame with coloured tape now. Every later step needs it.

> **POWER DURING STEPS 1–8**
>
> The ESP32 runs from the **USB cable**. The LiPo only powers the ESCs through the PDB. **H1 stays unplugged.** Never feed the board from USB and a BEC at the same time.

Check before the first LiPo connection

- [ ] Every ESC's thick red wire is on PDB **+**, thick black on **−**.
- [ ] Multimeter in beep mode between PDB + and −: **no beep** (no short circuit).
- [ ] The low-ESR capacitor is on the PDB battery pads, stripe (−) on the negative pad.

- [ ] Each motor's 3 wires go to its ESC (any order; Step 3 fixes direction).
- [ ] Props are **off**.

Step 2 Arm and calibrate the ESCs (sketch 1)

An ESC only obeys after it has **armed**: at power-up it must see a valid "stop" signal, then it plays its start-up tones and goes silent. Calibration then teaches each ESC that 1000 µs means "stopped" and 2000 µs means "full power", so all four start at the same throttle point. Sketch 1 sends a 50 Hz signal at start-up, the standard servo signal every ESC accepts.

2.1 First power-up: listen

1. Props **off**. ESP32 on USB, `1_motor_test.ino` uploaded, Serial Monitor open (115200, Newline). The menu and a STATUS block print.
2. Plug in the LiPo and listen. **Normal** = a melody, 3 short beeps (3 cells), one long beep, then **silence**. Silence means armed.
3. If the ESCs keep beeping instead, find the pattern in the table below (type `b` to print it in the Serial Monitor).

What you hear	Meaning	Do
Melody, 3 beeps, long beep, silence	normal: armed and ready	continue
One beep every ~2 seconds, forever	no valid signal	wiring check `p` (2.3); ESC ground on H6 pin 5; try another speed with `f`
Fast beeps (~4 per second), forever	throttle not at zero: "off" looks too high to the ESC	calibrate (2.2), or lower the off pulse with `o` and replug the LiPo
"beep-beep" pairs, about one per second	battery voltage wrong (LiPo flat, or ESC battery setting changed)	measure the LiPo: 11.1–12.6 V; charge it
"beep-beep", a melody, then repeating beep patterns	programming mode: the ESC saw full throttle too long at power-up	unplug the LiPo; redo the calibration
Nothing at all	no power to that ESC	check its thick wires on the PDB

2.2 Calibrate (all four ESCs at once)

The code does the timing for you: you only plug in the LiPo when the monitor says **NOW**.

1. LiPo **unplugged**, plug in your hand. Type `c` + Enter, then Enter again to start.
2. The signal goes to FULL (2000 µs) and the monitor counts **3... 2... 1...**
3. At **NOW: PLUG IN THE LIPO**, plug it in.
4. The code holds FULL for about 3–4 seconds, then drops to STOP by itself. You hear power-up tones, then **"beep-beep"** (full learned), then **3 short beeps + 1 long beep** (stop learned) = **saved**. If you hear "beep-beep" early, you may press **BOOT** or Enter to drop to STOP at once.
5. Unplug the LiPo, wait 5 seconds, plug it back in: melody, 3 beeps, long beep, silence.

If H5 (battery voltage) is wired, the sketch sees the exact moment you plug in and times it from there. It also refuses to start a calibration while the LiPo is already plugged in.

Result	Do
Only the normal start-up tones, no "beep-beep"	the ESC missed FULL: run `c` again and plug in exactly at NOW
A melody, then beep patterns that keep repeating	programming mode: unplug, run `c` again, plug in a little *later* after NOW

2.3 Wiring check with a multimeter (command `p`)

Use it when an ESC never arms. The sketch sends a safe "stop" pulse to one signal wire at a time; the other three get nothing. With the multimeter on DC volts, black probe on **H6 pin 5**:

- The active pin reads about **0.8 V**, the other three read **0 V**. Press Enter to move to the next pin.
- Measure again at each ESC plug (white/yellow wire against black/brown): it must also read about 0.8 V.
- 0 V on the active pin = broken trace or solder joint. Voltage on the wrong pin = wires in the wrong order. Right at H6 but 0 V at the ESC plug = plug reversed or bad crimp.
- LiPo unplugged, multimeter on beep mode: H6 pin 5 to PDB negative must **beep** (common ground).

Step 3 Test each motor alone (order + direction)

This step proves two things: the right motor answers each command, and it spins the right way. Props still **off**.

1. Stick a small flag of tape on each motor bell so you can see which way it turns.
2. Plug in the LiPo and wait for the start-up beeps to finish.
3. Type `1` + Enter. One motor spins slowly for 3 seconds at 1200 µs.
4. Looking **from above**: is it the **rear-right** motor (M1)? Does it spin **clockwise**?
5. Repeat with `2`, `3`, `4` and fill in the table.

Command	Should be	Spins?	Right arm?	Right direction?
`1`	M1 rear right, CW			
`2`	M2 front right, CCW			
`3`	M3 rear left, CCW			
`4`	M4 front left, CW			

Fixes

- **Wrong arm spins:** that ESC's signal wire is on the wrong H6 pin. Unplug the LiPo and move the signal wires until `1` = rear right, `2` = front right, and so on.
- **Wrong direction:** unplug the LiPo, then swap **any two** of the three wires between that motor and its ESC. Brushless motors have no + or −; the wire order only sets the direction.
- **Twitches but doesn't start:** raise the test throttle with `+` (try 1250) and retry.
- **Nothing at all:** is that ESC still beeping? Then it never armed: see the table in 2.1 and run the wiring check (2.3).

Move on only when all four rows are correct.

Step 4 All motors together + emergency stop

Check that the four motors run smoothly together and that you can stop them instantly. Props still **off**.

Stop method (sketch 1)	How	When to use it
Serial key	type any letter + Enter	normal stop at the laptop
BOOT button	press BOOT on the ESP32	you're next to the drone
Unplug the LiPo	pull the XT60 apart	always works, even if the code is frozen
Auto-stop	nothing to do	every test stops by itself after 3 s

1. Type `a`: all four ramp up together for 3 seconds. They should start at the same moment and sound the same.
2. Type `a` again and after 1 second press any key + Enter. All four must stop at once.
3. Type `a` again and press **BOOT**. All four must stop at once.
4. Type `s`: the motors run one after another, M1 → M4, as a last check of the order.
5. Press `+` up to 1300 and type `a`. The sound must rise smoothly with no stutter.

Practise all three stop methods now, while it's harmless, so your hands know them later.

4.1 Find the fastest signal speed your ESCs accept

The flight code works best with a fast signal (250 Hz), but some ESCs only accept slower ones. Find out now:

1. LiPo unplugged. Type `f`: the speed goes 50 → 100 → 200 → 250 → 400 Hz (then back to 50).
2. After each change, plug in the LiPo and listen: melody, 3 beeps, long beep, silence = accepted. Type `a`: all four must start together and run smoothly.
3. Unplug, press `f` for the next speed, repeat.
4. Write down the **fastest** speed that arms and runs smoothly. It goes into `ESC_FREQ_HZ` in sketch 4 (Step 10.1). The sketch always starts at 50 Hz again after a reset.

Step 5 Motor checks and adjustment

Most "the drone shakes / won't fly straight" problems are mechanical. Spend ten minutes here and you avoid hours of PID tuning later.

5.1 Mechanical checks (LiPo unplugged)

Check	How	If it fails
Motor screws not too long	Turn each bell by hand: it must spin freely, with no scraping or tight spot.	A screw that pokes into the motor touches the windings and can short and burn it. Use shorter screws or add a washer.
No up-down play in the bell	Pull the bell gently up and down.	Check the small clip (circlip) under the motor shaft is in place.
Motor tight on the arm	Try to twist the motor by hand.	Tighten all 4 screws evenly, with a drop of medium thread-lock if you have it.
Prop adapter straight	Spin by hand and look from the side: the adapter must not wobble.	Re-seat the collet adapter and tighten it evenly.
Wires safe	Bullet connectors fully pushed in and covered with heatshrink; no wire can reach a prop.	Zip-tie wires along the arms.

5.2 Find the start-up throttle and set the idle

When the drone is armed, the motors spin slowly at `MOTOR_IDLE`. That value must be just above the point where all four start reliably.

1. In sketch 1, press `-` until the test throttle is **1050**.
2. Type `a`. If not all four spin, press `+` once (25 µs) and type `a` again.
3. Note the lowest value where all four start every time, for example 1075.
4. Your `MOTOR_IDLE` for sketch 4 = that value + about 25 (for example 1100). Write it down.

5.3 Motor sync check

At 1150 µs and 1300 µs (`a`), all four should start together and sound identical. One motor starting late or sounding different means its ESC is calibrated differently: redo Step 2. If it persists, swap that ESC with another one: if the problem moves with the ESC, the ESC is the cause; if it stays on the motor, the motor is.

5.4 Temperature check

Run `a` at 1300 µs three times in a row, then unplug and touch each motor and ESC. With no props they should be barely warm. One clearly hotter than the others = bad solder joint, damaged winding or a rubbing bell.

Step 6 Props on: balance and tethered thrust test

First test with props. It checks that each prop **pushes air down** (lifts the drone) and that nothing vibrates badly. The drone must **not** be able to lift off.

6.1 Balance the props

Cheap props are often heavier on one blade. That makes the motor shake, and the shaking reaches the MPU6050 and makes the drone twitch.

1. Push a screwdriver or thin rod through the prop hub and rest it level between two glasses, so the prop turns freely.
2. Turn the prop horizontal and let go. If one blade always drops, it's heavier.
3. Fix it: lightly sand the **flat back face** of the heavy blade (never the edges), or add a small piece of clear tape near the tip of the light blade.
4. Repeat until the prop stays wherever you leave it. Do all four (and the spares).

6.2 Fit the props

- CW props (often marked **R**) on M1 and M4. CCW props on M2 and M3.
- The thick, rounded **leading edge** of each blade faces the direction the motor spins. The sharp edge trails.
- Writing on the prop faces **up** (to the sky).
- Tighten the prop nuts firmly. On self-tightening motors, each nut must tighten against its motor's spin.

6.3 Tie the drone down

- Strap or zip-tie the frame to a heavy board or a box of books (5 kg or more), or tie each arm with string to something heavy on the floor.
- Outside or in a room with nothing loose near the props. Glasses on if you have them.
- Stand behind the drone. Laptop at least 1 metre away.

6.4 Thrust test

1. Press `-` until the test throttle reads **1100**. Never go above **1250** in this step.
2. Plug in the LiPo, wait for the beeps.
3. Type `1`. Hold a light strip of paper about 10 cm **below** that prop, off to the side of the arm. It must be blown **downward**.
4. Repeat for `2`, `3`, `4`.
5. Type `a` at 1150–1200. The frame should feel pushed upward evenly, without strong shaking. All four should sound the same.
6. Unplug the LiPo.

> **HANDS AWAY FROM THE TOP**
> Air is sucked in from **above** the prop. Never hold paper, fingers or cables above a spinning prop: they get pulled in.

Problem	Cause	Fix
Paper pulled up / air blows upward	prop upside down, or CW/CCW prop on the wrong motor	flip it or swap it with the correct type
One arm shakes strongly	unbalanced or chipped prop, bent shaft	re-balance; swap with a spare to compare
Whole frame buzzes	loose motor screws or loose frame screws	tighten everything; redo 5.1

Part 3 — Radio and sensor

The receiver turns your stick movements into numbers; the MPU6050 tells the board which way is down. Both must be correct before the flight code can do anything useful. **Take the props off again** for this part.

Step 7 Receiver: connect, bind, set up, failsafe

The receiver sends all channels down **one wire** using the i-BUS protocol. Each channel is a number from 1000 to 2000, with 1500 at stick centre.

7.1 Connect the receiver to H2

Use the receiver's **i-BUS SERVO** port, not the CH1–CH10 pins and not i-BUS SENS (that one is for telemetry sensors).

Receiver pin	H2 pin	What it is
S (signal)	2 (I-BUS)	all channels, goes to GPIO 18
+	1 (5V)	power for the receiver
–	3 (GND)	ground

If the port has no labels: multimeter in beep mode, the pin that beeps with the **–** of the CH1 row is –, the one that beeps with its **+** is +, the last one is S.

Antennas: the FS-iA10B has two. Point them at 90° to each other (one horizontal, one vertical), away from the battery, motors and carbon parts. Never cut or bend the thin tips.

7.2 Transmitter basics (FS-i6X)

1. Switch on with all switches **up** and throttle **down** (the i6X warns otherwise).
2. Hold **OK** to open the menu. **System setup** and **Functions setup** are the two main menus. **UP/DOWN** move, **OK** selects, hold **CANCEL** to save and go back.
3. **System → Model select**: choose an empty model and give it a name (e.g. DRONE).
4. **System → Sticks mode**: **Mode 2** (throttle on the left stick).
5. **System → RX setup**: protocol **AFHDS 2A** (the FS-iA10B uses it).

7.3 Bind (pair) the receiver

Binding makes the receiver listen only to your transmitter. You do it once; it stays bound after power-off.

1. Transmitter **off**.
2. Put the **bind cable** (the short loop plug that came with the receiver) into the receiver's **B/VCC** port, or press and hold the receiver's bind button if yours has one.
3. Power the ESP32 by USB. The receiver gets power through H2 and its LED **blinks fast** = bind mode.
4. Hold the transmitter's **BIND KEY** and switch it on (or use **System → RX bind**). The screen shows "RX Binding...".
5. When the receiver LED changes to slow blinking or solid, binding is done.

6. Remove the bind cable, switch everything off, then on again (transmitter first). A **solid LED** = bound and receiving.

Receiver LED	Meaning
Solid on	bound and receiving the transmitter
Fast blinking	bind mode (bind cable in)
Slow blinking	no signal: transmitter off, out of range, or not bound to this transmitter/model

7.4 Channel setup

1. **System → RX setup → Output mode**: set **Serial** to **i-BUS**. Hold CANCEL to save.
2. **Functions setup → Aux. channels**: **Channel 5 = SwA** (arm / kill switch) and **Channel 6 = SwC** (flight mode: MANUAL / ALT HOLD / LAND).
3. **Functions setup → Reverse**: all *Normal* for now; you fix directions in 7.5.
4. **Functions setup → End points**: leave 100 % for now.

7.5 Test with sketch 2

Upload `2_receiver_test.ino`, open the Serial Monitor, switch the transmitter on. Each line shows ROLL, PITCH, THR, YAW, CH5 with ARM / SAFE, CH6 with MANUAL / ALT HOLD / LAND, and CH7–CH10. Move each control:

Control	Channel	Must go to	If it goes the other way
Right stick → right	CH1 (ROLL)	about 2000	Reverse CH1
Right stick → forward (away from you)	CH2 (PITCH)	about 2000	Reverse CH2
Left stick → up	CH3 (THR)	about 2000	Reverse CH3
Left stick → right	CH4 (YAW)	about 2000	Reverse CH4
Flip SwA both ways	CH5	about 1000 one way, about 2000 the other	check Aux. channels = SwA
Move SwC through its 3 positions	CH6	about 1000, 1500, 2000	check Aux. channels = SwC

Your armed position is whichever SwA position makes sketch 2 print `CH5 … ARM` (about 2000); the other one prints `SAFE`. SwC prints `MANUAL` (about 1000), `ALT HOLD` (1500) and `LAND` (2000). Reverse CH5 or CH6 if you prefer the other direction. Mark ARM and the three SwC positions on the transmitter with tape. Then fine-tune:

- Centred sticks (roll, pitch, yaw) should read **1490–1510**. Correct with **Functions → Subtrim**.
- Throttle fully down should read **1000–1010**; fully up about 2000. Correct with **End points**.

7.6 Failsafe (very important)

Failsafe decides what the receiver sends if it loses the transmitter. With the version 3 flight code the drone should **land by itself** (Step 15), so the receiver must report "SwC on LAND, throttle down, sticks centred, still armed":

Channel	Failsafe	Position to hold while saving	Why
CH1, CH2, CH4	**On**	sticks centred	the drone levels instead of flying on
CH3	**On**	throttle fully down	never full throttle
CH5 (SwA)	**Off**	–	keeps ARM, so the drone can land instead of falling
CH6 (SwC)	**On**	SwC on LAND	the flight code lands

1. **Functions setup → Failsafe**: select a channel, put the stick or switch in position, choose **On** (Off for CH5), **hold CANCEL** to save. Repeat for each channel.
2. Test: with sketch 2 running, SwA on ARM, SwC on MANUAL, switch the transmitter **off**. Within about a second the monitor must show THR about 1000, CH1/2/4 about 1500, **CH5 still ARM** and **CH6 LAND**, or **SIGNAL LOST**. Both are handled by the flight code: in the air it lands, on the ground it stops the motors.

> **COMING FROM VERSION 2 OF THIS GUIDE?**
> Version 2 set CH5 to SAFE in the failsafe. With that setting a radio loss stops the motors and the drone falls. Change CH5 to Off and add CH6 as above.

7.7 Range check

With sketch 2 running on the laptop, ask a friend to watch the screen while you walk away with the transmitter. The values must keep updating smoothly to at least 50 m on the ground, with the antennas set as in 7.1. If it drops out early, check the antennas.

Step 8 MPU6050: mount, orientation, level calibration

The MPU6050 is the drone's sense of balance: it measures how fast the frame rotates (gyro) and which way is down (accelerometer). The code must know exactly how the chip sits on the frame; otherwise it corrects in the wrong direction and flips the drone.

8.1 Mount it

The U2 footprint was drawn for an MPU9250, but its first 8 pins match the GY-521 (MPU6050) module exactly. Pins 9 and 10 stay empty.

U2 pin	PCB label	GY-521 pin
1	VCC	VCC
2	GND	GND
3	SCL	SCL (GPIO 8)
4	SDA	SDA (GPIO 9)
5 / 6	EDA / ECL	XDA / XCL (unused)
7	AD0	AD0 (tied to GND → address 0x68)
8	INT	INT (unused)

Fix the module **flat and firm** with a drop of hot glue or double-sided foam tape. A loose sensor picks up vibration and the drone shakes in flight.

The GY-521 usually comes with its pin header **loose in the bag**: solder it to the module first. Pins only pushed through the holes don't make contact, and the sensor won't answer. With USB plugged in, the small LED on the module must light.

If sketch 3 prints **MPU6050 NOT FOUND**, it also measures the SDA/SCL voltages, scans the bus and prints the most likely cause (short, no power, pins not touching, AD0).

8.2 Note the arrow

The GY-521 has an **X** arrow printed on it. Looking from above, note where it points, then set it at the top of sketch 3:

```
const int  IMU_ROTATION    = 0;     // 0 forward, 90 right, 180 backward, 270 left
const bool IMU_UPSIDE_DOWN = false; // true only if the chip faces the ground
```

8.3 The three checks

Upload `3_imu_test.ino` and keep the drone perfectly still for the first 4 seconds (gyro calibration). Then:

Move the drone	Watch	Must become	Pass?
Tilt the **right** side down	ROLL	positive (+)	
Lift the **nose** up	PITCH	positive (+)	
Turn the nose to the **right** (clockwise from above)	YAW rate, while turning	positive (+)	

If a check fails, try the next `IMU_ROTATION` (0 → 90 → 180 → 270), upload, and repeat. If none of the four passes all three checks, set `IMU_UPSIDE_DOWN = true` and try the four again. **Write down the values that pass.**

8.4 Level calibration

Put the drone on a surface that is truly level (check with a phone level app). Type `l` + Enter and don't touch it for 3 seconds. The monitor prints two lines like:

```
#define ACC_ROLL_TRIM     1.35f
#define ACC_PITCH_TRIM    -0.80f
```

Copy them; they replace the two `ACC_..._TRIM` lines in sketch 4's `config.h` (section 1), together with the `IMU_ROTATION` and `IMU_UPSIDE_DOWN` values that passed 8.3. Afterwards ROLL and PITCH read close to 0.0 on the level surface. Without this, the drone drifts sideways in every hover.

Step 9 Power for flight + final wiring check

In flight there is no USB cable, so the board needs 5 V from the battery side. It comes in through **H1** and powers the ESP32, the receiver and (through the ESP32's 3.3 V regulator) the MPU6050.

Option	How	Notes
A. One ESC's BEC	red + black wires of ONE ESC's servo lead → H1 pin 1 (5V) and pin 2 (GND). Its white signal wire still goes to H6.	Only if the ESC has a BEC (label says "BEC 5V"). Pull the 3 pins out of the housing to split them.
B. Separate UBEC (better)	a 5 V / 3 A UBEC soldered to the PDB battery pads, output to H1	Cleaner power, less noise on the gyro, no brown-outs.

The other three ESC red wires stay **disconnected**. Never connect two BECs together.

> **USB AND H1 NEVER AT THE SAME TIME**
>
> Bench tests with the Serial Monitor: H1 unplugged, USB in. Flight: USB out, H1 plugged. Measure the polarity of the 5 V with a multimeter **before** plugging it into H1.

Final wiring check

- [ ] H6: M1–M4 signal wires on pins 1–4, ESC ground on pin 5
- [ ] H2: receiver i-BUS SERVO, S / + / – the right way round
- [ ] U2: MPU6050 in pins 1–8, fixed flat and firm
- [ ] H1: 5 V from one BEC or a UBEC, polarity checked
- [ ] H5 (battery voltage): PDB + to pin 1, PDB – to pin 2
- [ ] Battery strapped tight, centre of gravity in the middle of the frame
- [ ] All wires tied away from the props; receiver antennas clear
- [ ] LiPo alarm on the balance plug (the board can't warn you in flight)

Part 4 — Flying

The flight code keeps the drone level by itself; you only steer. First you prove on the bench that it corrects in the right direction, then you hop, hover, and tune.

Step 10 The flight controller code and bench test

`4_flight_controller` flies in **angle mode**: with the sticks centred the drone holds itself level, and the right stick tilts it up to 30° so it moves. This is the easiest mode for a first drone. Fly the first flights in **MANUAL** (SwC on MANUAL); Part 5 adds ALT HOLD and LAND.

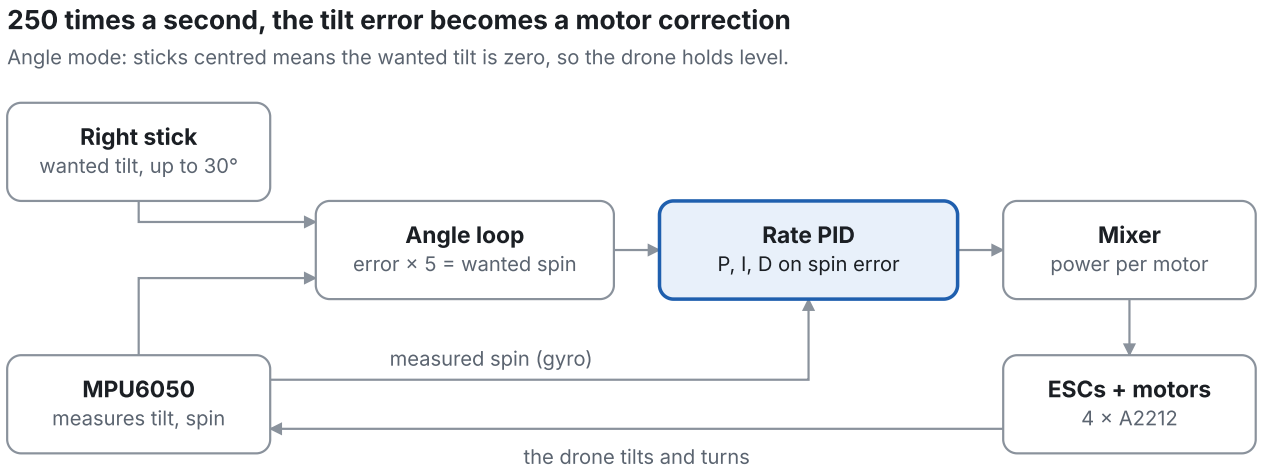


Figure 3 — The control loop. P reacts to the error now, I removes a slow lean, D brakes the motion so it doesn't overshoot.

10.1 Edit config.h (the settings file of sketch 4)

Setting	Put in	From
`IMU_ROTATION`, `IMU_UPSIDE_DOWN`	the values that passed the 3 checks	Step 8.3
`ACC_ROLL_TRIM`, `ACC_PITCH_TRIM`	the 2 numbers sketch 3 printed	Step 8.4
`ESC_FREQ_HZ`	the fastest speed that worked (250 if it did; if only 50 Hz worked, use 50 and lower `P_ROLL` and `D_ROLL` about 30 %)	Step 4.1
`MOTOR_OFF`	1000, or the off pulse that let the ESCs arm (`o`)	Step 2.1
`MOTOR_IDLE`	your idle value (about 1100)	Step 5.2
`VBAT_CAL`	multimeter volts ÷ printed volts (start at 1.00)	Step 13.4

Leave the PID gains as they are for the first flights. Every setting is explained in `config.h` itself and in `docs/02-firmware-overview.md`.

10.2 Arm and disarm

Action	How	What happens
Arm	SwC on **MANUAL**, throttle fully down, then SwA to **ARM**	1 beep, LED blue, motors spin slowly (idle)
Fly	raise the throttle	the stabiliser works above the idle zone
Disarm / kill	SwA to **SAFE**	2 beeps, all motors stop instantly, even in the air

It **refuses to arm** (3 fast beeps, the Serial Monitor says why) if the throttle isn't down, SwC isn't on MANUAL, the drone isn't level, the radio is off, the battery is too low, or the MPU6050 is missing. If SwA was already on ARM at power-on, flip it to SAFE and back. It **stops the motors by itself** if the drone tilts past 70° (a crash), the MPU6050 stops answering, the radio is lost while on the ground, or the throttle stays down for 15 s on the ground. In the air, a radio loss or an empty battery make it **land** (Step 15).

10.3 Bench test (props OFF, drone held in your hand)

USB in, H1 unplugged, sketch 4 uploaded, Serial Monitor open. Leave the drone flat and untouched for the first **8 seconds** (gyro calibration).

1. After the long beep the monitor shows `READY | roll +0.3 pitch -0.2 | THR 1000 SwA SAFE SwC MANUAL | h +0.04 m | 12.53 V 98%` (or `WAIT … <- reason`), and the LED is green. Tilt the drone: roll and pitch follow. Compare the voltage with a multimeter and set `VBAT_CAL` if needed (Step 13.4).
2. Plug in the LiPo. Throttle down, arm → `ARMED`, 1 beep, LED blue, all four motors idle slowly. If one doesn't turn, raise `MOTOR_IDLE` by 10; if they spin fast, lower it.
3. Disarm → all stop at once. Arm again.
4. Raise the throttle to about one third. Hold the frame **firmly** and do the correction checks below, listening to the motors.

You do	These motors must speed up	Why	Pass?
Tilt the right side down	M1 + M2 (right side)	pushes the right side back up	
Tilt the nose down	M2 + M4 (front)	lifts the nose back up	
Twist the nose to the right	M1 + M4 (the CW motors)	turns the nose back left	

> **DO NOT FLY IF ANY ROW IS BACKWARDS**
>
> A drone that corrects the wrong way flips the instant it leaves the ground. Go back to Step 8 (IMU settings), then Step 3 (motor order).

5. Throttle back down (motors idling), then switch the transmitter **off**: the motors must stop within about a second (receiver failsafe or radio lost, on the ground).
6. Unplug the LiPo.

Switch to flight power: unplug USB, plug H1 (Step 9). From now on, plugging in the LiPo boots the board. Keep the drone still until the **long beep** (about 10 seconds) after plugging it in.

Step 11 First hover

The first flight is a series of short hops, not a flight. The goal is to see the drone lift straight up and sit there.

- [ ] LiPo fully charged (12.6 V), LiPo alarm plugged in
- [ ] Props tight, correct CW/CCW, no chips
- [ ] Step 10.3 correction checks all passed
- [ ] Open field with grass, nobody within 15 m, little wind

11.1 Hop test

1. Drone on flat ground, front pointing away from you. Stand **5 m behind** it.
2. Plug in the LiPo, step back, wait for the **long beep** (about 10 seconds) without touching it.
3. Throttle down, arm: the motors idle.
4. Raise the throttle **slowly** until the drone gets light and lifts a few centimetres. Remember that stick position: it's your hover point (usually near the middle).
5. Land straight away: throttle down, disarm.

> **ABORT IMMEDIATELY**
> Disarm at once if it flips or tilts hard to one side the moment it gets light. A correction is backwards: redo Step 10.3.

11.2 Short hover

Lift to about 1 m and hold 10–20 seconds. Small corrections with the right stick are normal. Land, disarm, unplug, and touch the motors and ESCs: warm is fine; too hot to hold is not.

Step 12 PID tuning

Change **one value at a time**, by about 15 %, then fly again. The gains are in `config.h`; re-upload over USB with H1 unplugged. Write each change in the flight notebook (Part 6) and keep the flight log files (Step 16) so you can compare and go back.

What you see	Most likely cause	Change
Fast buzzing or shaking, hot motors	D too high (or unbalanced props)	lower `D_ROLL` (18 → 15); re-check Step 6.1
Fast wobble right after a stick move	P too high	lower `P_ROLL` (1.3 → 1.1)
Lazy, wanders, slow to come back level	P too low	raise `P_ROLL` (1.3 → 1.5)
Slow rocking (about once per second)	level pull too strong, or I too high	lower `LEVEL_GAIN` (5 → 4) or `I_ROLL`
Leans one way for a few seconds, then catches up	I too low	raise `I_ROLL` (0.04 → 0.05)
Nose slowly turns by itself	yaw too weak	raise `P_YAW` (4 → 5)
Always drifts the same way in calm air	level not calibrated, or weight off-centre	redo Step 8.4; centre the battery
Too sensitive / not sensitive enough on the sticks	stick range	`MAX_ANGLE_DEG` (30) and `MAX_YAW_RATE` (150)

Roll and pitch share the same gains because the frame is symmetric, so one change fixes both. **Land when the LiPo alarm beeps**, or after about 5 minutes for the first flights.

Part 5 — Version 3: smarter and safer

Version 3 of the flight code adds altitude hold, automatic landing, battery management with a percentage, buzzer and LED status, and a flight recorder. Do these steps only after Part 4: the drone must already hover well in MANUAL.

Step 13 Version 3 setup: files, buzzer, LED, battery

13.1 What changed

Feature	What it does for you	Step
Flight modes on SwC (CH6)	MANUAL (as before), **ALT HOLD** (holds the height by itself), **LAND** (lands and stops)	14, 15
Automatic landing	radio lost, battery empty or barometer lost in the air → the drone lands instead of falling	15
Battery management	percentage in flight, LOW warning, CRITICAL automatic landing, no take-off with an empty pack	13.4
Buzzer + LED	you hear and see what the drone does: armed, mode, warnings, refusals	13.3
Flight recorder	50 measurements per second, saved after landing, graphs on your laptop	16
GitHub project	code, documentation and automatic tests online for your friend and the judges	17

The sketch is now split into files. **You only edit** `config.h`; the other `.h` files are the modules (radio, motors, PID, MPU6050, BMP280, height, battery, buzzer/LED, log). Open `4_flight_controller.ino`: the Arduino IDE shows every file as a tab.

13.2 Copy your values into config.h

The values from Step 10.1 (IMU orientation and trims, `ESC_FREQ_HZ`, `MOTOR_OFF`, `MOTOR_IDLE`, your PID values from Step 12) now live in `config.h`. Check the Tools menu once more: **PSRAM = OPI PSRAM** (room for 16 minutes of flight log) and **Partition Scheme = 16M Flash (3MB APP/9.9MB FATFS)** (the log is saved there). The very first start after the upload can take about 30 s while the log storage is prepared.

13.3 Buzzer and LED

The PCB has no buzzer header, so the code uses the free **H3** (GPS) connector. Use an **active buzzer module marked "high level trigger"**.

Buzzer module	H3 pin
VCC	1 (5 V)
I/O (signal)	3 (GPIO 17)
GND	4

BEEPS ALL THE TIME?

A "low level trigger" module on 5 V never switches off with the ESP32's 3.3 V. Power it from 3.3 V and set `BUZZER_ACTIVE_HIGH false`, or buy the high-level type. No buzzer: `BUZZER_PIN -1`.

Beep	Meaning
1 long	ready to arm (after power-on, after the log is saved)
1 short	armed, or mode accepted
2 short	disarmed
3 fast	refused: the Serial Monitor says why
1 every second	battery LOW
fast chirps	battery CRITICAL
beep-beep	landing
chirp every 2 s	"find me" after a crash or radio-loss landing

LED (on the DevKit)	Meaning
yellow	starting: keep the drone still
green	ready to arm
green blinking	not ready (status line says why)
blue	armed, MANUAL
purple	ALT HOLD
orange blinking	LAND
yellow flashes	battery LOW
red blinking	battery CRITICAL / MPU6050 missing
white	saving the log: **don't unplug**

13.4 Battery: calibrate, then let the drone watch it

1. LiPo in (props off), Serial Monitor open, type `i`. Measure the battery at the XT60 with a multimeter.
2. Set `VBAT_CAL = multimeter volts ÷ drone volts` in `config.h` (for example 12.31 ÷ 12.18 = 1.011) and upload again.

> **WHY THE DRONE SHOWS A "RESTING" VOLTAGE**
> Under load a LiPo's voltage drops (it "sags") and comes back after landing. A pack at 60 % can read like 30 % in flight. We have no current sensor, but we know how hard the motors work. Just before arming the battery rests, so in the first seconds of flight the code learns how much the voltage drops per unit of motor load, and from then on adds that drop back. The percentage and the warnings use this *estimated resting voltage*.

State	When (for 5 s)	What the drone does
LOW	resting estimate below 3.75 V/cell (~25 %)	keeps flying; beep every second, yellow flashes: land soon
CRITICAL	below 3.65 V/cell (~7 %), or measured below 3.30 V/cell	in the air: **lands by itself**, SwC is locked, you can still steer
No arm	below 3.75 V/cell at rest	refuses to arm (3 beeps): charge the pack

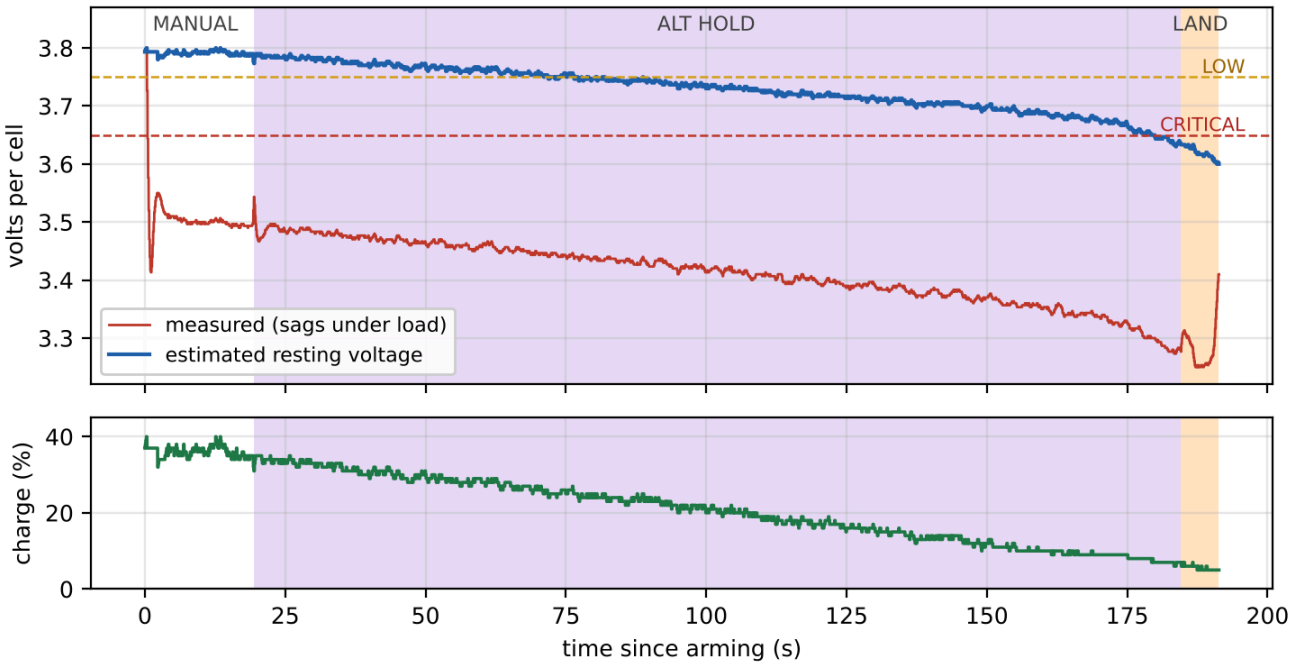


Figure 4 — Simulated flight with a pack started at 40 %. Red: measured voltage, low because of the sag. Blue: estimated resting voltage. LOW came at about 25 % real charge, the automatic landing at about 8 %; it landed with about 200 mAh left.

Step 14 ALT HOLD: the drone holds its height

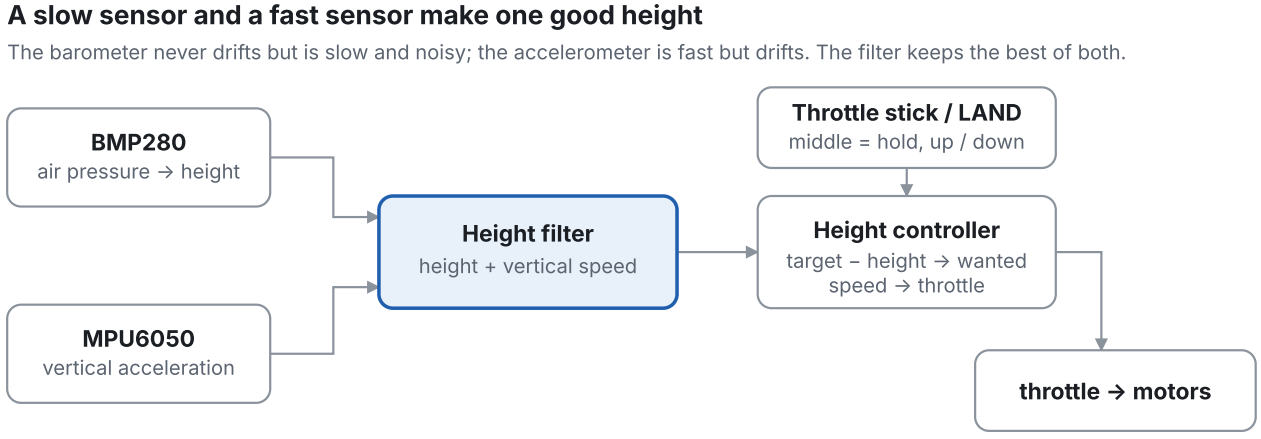


Figure 5 — ALT HOLD. The controller starts from the hover throttle it learned while you flew, so switching modes is smooth.

14.1 Set up the switch and the sensor

1. Transmitter: **Functions setup → Aux. channels → Channel 6 = SwC**.
2. Sketch 2: CH6 must show `MANUAL`, `ALT HOLD` and `LAND` in the three positions. Mark the positions on the transmitter.
3. Cover the BMP280 with a small piece of **open-cell foam** (like the foam of headphone pads) and keep it out of the sun: wind from the props and light disturb it.
4. Bench check: lift the drone 1 m by hand (props off). The status line `h` follows within about 0.2 m. `z` sets zero.

14.2 Fly it

1. Take off in **MANUAL** to 1.5–2 m and hover steadily for 3 s (the code learns the hover throttle).
2. SwC to the **middle**: 1 short beep, LED purple. (3 fast beeps = refused: below 0.5 m or no barometer.)
3. Put the throttle stick in the **middle**: the drone holds its height. Stick up = climb (max 0.8 m/s), down = descend (max 0.6 m/s); back to the middle = hold the new height.
4. Back to MANUAL: first put the throttle stick where the drone hovers, then flip SwC.

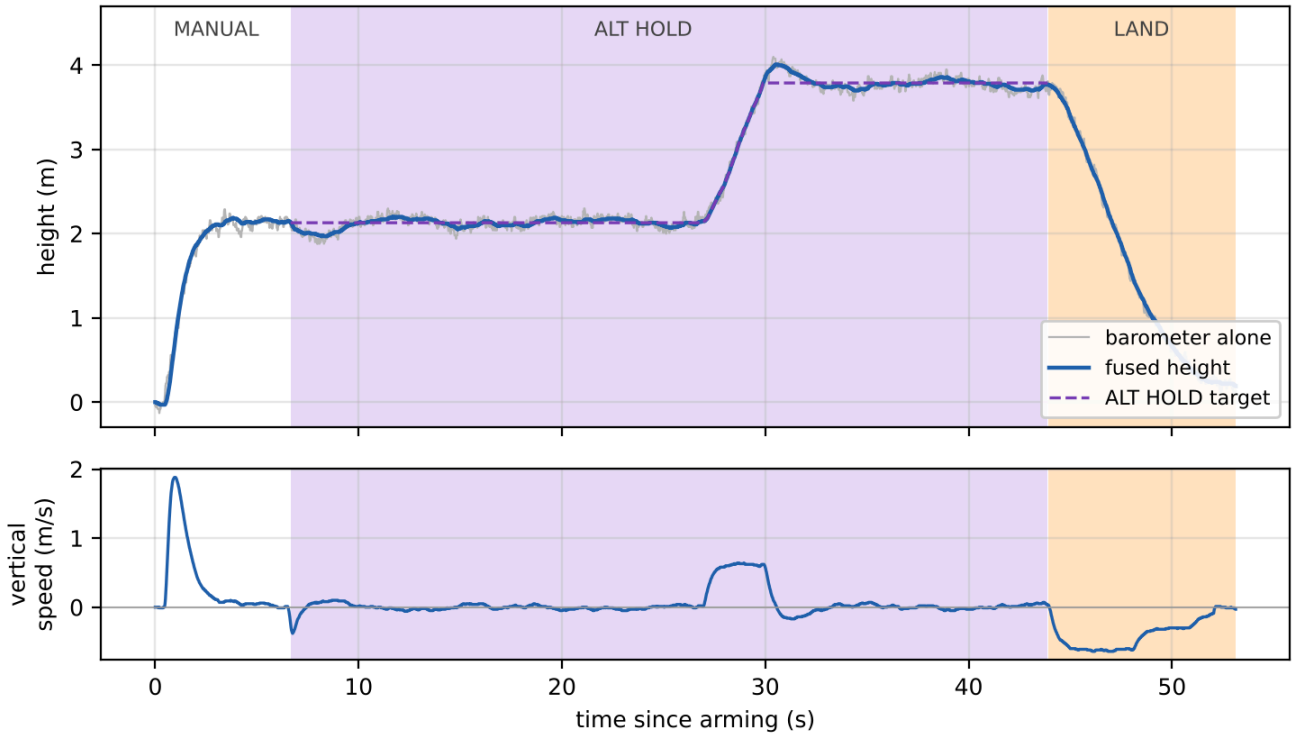


Figure 6 — Simulated flight: take-off in MANUAL, ALT HOLD at 2 m (±0.25 m), 3 s of climb with the stick, ALT HOLD at 3.8 m, then LAND: 0.6 m/s, then 0.3 m/s below 1.5 m, touchdown at 0.1 m/s.

14.3 Tune it (only after the attitude PID is good)

What you see in ALT HOLD	Change in config.h
slow up-down "breathing" over several seconds	lower `VEL_I` (40 → 25); foam on the BMP280
fast bouncing, about once per second	lower `VEL_P` (80 → 60)
sinks or climbs for seconds after switching	hover steadily 3 s in MANUAL before switching
lazy, drifts 0.5 m before correcting	raise `ALT_P` (0.8 → 1.0)
jumps when the motors spin up, or near the ground	prop wind on the barometer: more foam, move the sensor

Step 15 LAND and automatic landings

15.1 The LAND switch

SwC to **LAND** (LED orange, beep-beep): the drone comes down at 0.6 m/s, then 0.3 m/s below 1.5 m, you can still steer with the right stick, and it **stops the motors by itself** on the ground. To cancel, flip SwC

back. Holding the throttle stick fully down in ALT HOLD does the same. LAND while the drone sits on the ground simply stops the motors.

> **HOW DOES IT KNOW IT HAS LANDED?**
> It cannot see the ground. It notices that the controller keeps asking for less throttle but the drone does not move any more: speed near zero, throttle 40 µs below the descent average, no rotation, for 1 second (the method of the INAV autopilot). In the simulator the motors stop about 1 s after touchdown. Safety nets: throttle at the minimum for 3 s, or 30 s spent below 1.5 m.

15.2 What lands the drone by itself

Situation	In the air	On the ground
radio lost (no frames for 0.25 s)	LAND, sticks centred; radio back: move SwC to take over	motors stop
receiver failsafe values (15.3)	LAND	motors stop
battery CRITICAL	LAND, SwC locked, steering works	warning only
barometer lost in ALT HOLD	LAND without barometer: a firm landing (throttle 40 µs under hover, motors off after 20 s)	–
tilt over 70° (crash, motor failure)	motors stop + "find me" chirp	motors stop
SwA to SAFE	motors stop at once, always	

15.3 Receiver failsafe

The receiver must report "SwC on LAND, throttle down, sticks centred, still armed" when it loses the transmitter: **CH1, CH2, CH4 On (centred), CH3 On (down), CH5 Off, CH6 On (LAND)**, exactly as in Step 7.6. Check it again now with sketch 2.

> **STILL THE VERSION 2 SETTING?**
> With CH5 = SAFE in the failsafe, a radio loss stops the motors and the drone falls. That is acceptable only very low over grass.

15.4 Test in this order

- [ ] Props off: arm, motors idling, SwC → ALT HOLD: refused (3 beeps). SwC → LAND: motors stop.
- [ ] Props off: arm, motors idling, transmitter off: motors stop within about 1 s.
- [ ] Props on: LAND from 2 m, then from 5 m. The motors stop by themselves within about 2 s.
- [ ] ALT HOLD at 3 m over grass, transmitter off: the drone lands and stops. Switch the transmitter on again only after the landing.
- [ ] Once, with a nearly empty pack at low height: let CRITICAL land the drone, so you know how it sounds.

Step 16 Flight log: see what the drone did

While armed, the drone records 50 lines per second (angles, wanted angles, height, target, vertical speed, throttle, the 4 motors, battery, mode and events) in its PSRAM: about 16 minutes. After disarming it saves them to flash (LED white, about 1 s per minute of flight), then gives the long beep. **Wait for the long beep before unplugging.** The 20 newest logs are kept.

16.1 Download and plot

Install Python 3, then once: `pip install -r tools/requirements.txt`. After a flight: long beep, unplug the LiPo, plug USB, close the Serial Monitor, then:

```
python tools/flight_log.py list     --port COM10
python tools/flight_log.py download --port COM10 --out flight1.csv
python tools/flight_log.py summary  flight1.csv
python tools/flight_log.py plot     flight1.csv
```

Without Python: type `d` in the Serial Monitor and copy the lines between `BEGIN_LOG` and `END_OF_LOG` into a `.csv` file for a spreadsheet.

Graph	Good	Problem
height	smooth, close to the target in ALT HOLD	slow waves or bouncing: Step 14.3
angles	solid line follows the dashed wanted line	fast wiggles: P or D too high; slow waves: too low
motors	four lines close together	one always higher: heavy corner or weak motor; lines at 2000: not enough power
battery	blue (resting) smooth and slowly falling	big gap red–blue: tired or small pack

16.2 Serial Monitor commands (only while disarmed)

Type	Does	Type	Does
`h`	help	`d` / `d3`	print the newest log / log 3 as CSV
`i`	sensors, battery, hover throttle, last flight summary	`X`	erase all logs
`s`	status lines on / off	`z`	height 0 here
`l`	list logs		

After each flight a short summary is printed: duration, highest point, lowest voltage, and every event with its time (ALT HOLD at 2.1 m, LAND (switch), DISARMED: landed...).

Step 17 Put the project on GitHub

The project folder `esp32-drone-fc` is ready for GitHub: code, documentation (`docs/`, with the same chapters in more depth), the log tool, a simulator, and automatic tests.

```
esp32-drone-fc/
├── firmware/   1_motor_test  2_receiver_test  3_imu_test
│               4_flight_controller (edit config.h)  extras/bench_test
├── tools/      flight_log.py  requirements.txt  sim/ (simulator)
├── docs/       10 chapters + resources + this PDF
├── hardware/   schematic, PCB view, pin map
└── README.md   LICENSE  CHANGELOG.md  .gitignore  .github/
```

17.1 First push

Create an empty repository on github.com (**+ → New repository**, name `esp32-drone-fc`, Private, no README). Then in Git Bash inside the folder:

```
git config --global user.name  "Your Name"
git config --global user.email "your GitHub e-mail"
git init
git add .
git commit -m "Flight controller v3"
git branch -M main
git remote add origin https://github.com/YOUR-USERNAME/esp32-drone-fc.git
git push -u origin main
```

GitHub Desktop works too: *File → Add local repository → Create → Publish repository*.

17.2 Work together

1. **Settings → Collaborators → Add people**: your friend's username. He accepts, then `git clone`s the project.
2. Each change on its own branch: `git switch -c tune-alt-hold`, edit, test, `git commit`, `git push -u origin tune-alt-hold`.
3. On GitHub: **Compare & pull request**. Your friend reviews the changed lines, comments, approves; then **Merge**.
4. The **Actions** tab runs the simulator (17 flight scenarios) and builds every sketch on each push: a green tick means nothing broke.

> **WHAT YOUR FRIEND SHOULD CHECK FIRST**
>
> The pin table in `config.h` against his schematic, the buzzer on H3, the battery divider values, the power rules, and the list of known PCB issues in `docs/01-hardware.md` (for example H4 shares the upload pins GPIO 43/44).

17.3 The simulator

`bash tools/sim/run_tests.sh` (Linux, macOS, or Windows with WSL / Git Bash and g++) compiles the real flight code on your PC with a virtual drone, sensors, radio and battery, and flies 17 scenarios: take-off, ALT HOLD, LAND on a slope, battery landing, radio loss, receiver failsafe, kill switch, crash, barometer failure, logging. Run it after every code change; it must end with `ALL SCENARIOS PASSED`.

Part 6 — Reference

Keep this part printed in your drone box.

6.1 LiPo battery care

Situation	Rule for your 3S 3300 mAh
Charging	Balance-charge only, at **1C = 3.3 A** or less, in a LiPo bag, never unattended. Full = 4.20 V per cell (12.6 V).
Flying	The drone warns at about 25 % (LOW) and lands by itself at about 7 % (CRITICAL). Never below 3.0 V per cell; that permanently damages it.
After flying	Let it cool before charging.
Storage (more than 2 days)	Charge or discharge to **3.8 V per cell** (11.4 V) with the charger's Storage mode. Keep cool, in the bag.
Damage	Puffed, dented or hot after a crash: stop using it. Put it outside on sand or concrete, away from anything that burns.

6.2 Pre-flight checklist

- [ ] LiPo charged, alarm plugged in
- [ ] Props: right type on each motor, tight, no chips
- [ ] Motor screws tight, bells spin freely
- [ ] Battery strapped, centred
- [ ] Wires and antennas clear of props
- [ ] USB unplugged, H1 plugged
- [ ] Transmitter on first, SwA SAFE, SwC MANUAL, throttle down
- [ ] Correct model selected on the i6X
- [ ] Area clear, people 15 m away
- [ ] Plug LiPo, keep still until the long beep, LED green
- [ ] Arm, short hop, then fly
- [ ] After landing: SwA SAFE, wait for the long beep (log saved), unplug LiPo, then transmitter off

Flight notebook

Date	Flight	Battery V before / after	Settings changed	What happened

6.3 Troubleshooting

Problem	Check first
Menu doesn't print / commands do nothing	Serial Monitor at 115200 and Newline; USB CDC On Boot = Disabled with the COM port
Upload stuck at `Connecting....`	hold BOOT, tap RST, release BOOT; COM port; data cable; nothing plugged into H4
Serial Monitor shows garbage	115200 baud; USB CDC On Boot = Disabled; cable in the COM port
Board crashes right after upload	PSRAM must be *OPI PSRAM* for the N16R8
ESC beeps every ~2 s forever	no valid signal: wiring check `p`, ESC ground on H6 pin 5, slower speed with `f` (Step 2.1)
ESC beeps fast (~4 per second) forever	throttle not at zero: calibrate (Step 2.2) or lower the off pulse with `o`
ESC beeps "beep-beep" pairs	battery voltage wrong: charge the LiPo (11.1–12.6 V)
One motor starts later than the others	redo ESC calibration (Step 2); swap ESCs to find the culprit
Motor stutters or won't start under load	bad solder joint or loose bullet connector on one of its 3 wires
`SIGNAL LOST` / `NO RADIO`	receiver LED solid? S on H2 pin 2? Output mode = i-BUS? Port = i-BUS SERVO?
A stick moves the wrong way	Reverse menu for that channel (Step 7.5)
`MPU6050 NOT FOUND` / I2C "nack" errors	sketch 3 prints an I2C check with the most likely cause. Usual causes: header pins not soldered to the GY-521, module in the wrong holes (VCC must go in U2 pin 1), GY-521 LED off = no power
No `IMU_ROTATION` passes all 3 checks	set `IMU_UPSIDE_DOWN = true` and try the 4 rotations again
`NO ARM: throttle not fully down`	throttle fully down, then SwA to SAFE and back to ARM
`NO ARM: drone not level`	flat ground; redo level calibration if it reads off on flat ground
`NO ARM: SwC (CH6) not on MANUAL`	SwC to MANUAL; check Aux. channels CH6 = SwC with sketch 2
`NO ARM: battery too low, charge it`	charge the pack; check `VBAT_CAL` (Step 13.4)
3 fast beeps when flipping SwC	ALT HOLD refused: climb above 0.5 m, check the BMP280 with `i`; or SwC locked during a battery landing
Buzzer beeps all the time	"low level trigger" module: Step 13.3

Climbs or sinks in ALT HOLD	throttle stick not in the middle; hover 3 s in MANUAL first; foam on the BMP280 (Step 14)
Motors keep running after a LAND	SwA to SAFE; then check the log: vertical speed and throttle at touchdown (Step 15.1)
`Log flash: NOT AVAILABLE` / `small RAM buffer`	Partition Scheme 16M Flash (3MB APP/9.9MB FATFS) / PSRAM OPI (Step 1.5)
`flight_log.py` : no answer	drone disarmed? Serial Monitor closed? right port (`python tools/flight_log.py ports`)?
Board resets when motors spin up	weak 5 V: use a UBEC on H1 (Step 9, option B)
Flips on take-off	a correction is backwards: Step 10.3, then Steps 8 and 3
Shakes in hover	prop balance (6.1), loose MPU6050 (8.1), then lower D (Step 12)

6.4 Pin map and radio channels

Function	Connector	ESP32-S3
M1 / M2 / M3 / M4	H6 pins 1–4	GPIO 10 / 11 / 12 / 13
ESC ground	H6 pin 5	GND
Receiver i-BUS	H2 pin 2	GPIO 18
MPU6050 SDA / SCL	U2 pins 4 / 3	GPIO 9 / 8
BMP280 (same I²C bus)	U3	0x76 / 0x77
Battery voltage	H5 (10k / 3.3k)	GPIO 1
Buzzer	H3 pin 3	GPIO 17
RGB LED	on the DevKit	GPIO 48
5 V in (flight)	H1	5V0
Emergency stop (sketch 1)	BOOT button	GPIO 0

Channel	Control (Mode 2)	Used for
CH1	right stick left/right	roll
CH2	right stick forward/back	pitch
CH3	left stick up/down	throttle
CH4	left stick left/right	yaw
CH5	SwA	arm / kill switch
CH6	SwC	MANUAL / ALT HOLD / LAND

6.5 Glossary

Word	Meaning
3S	three LiPo cells in series: 11.1 V nominal, 12.6 V full
Arm / disarm	allowing / forbidding the motors to spin. Disarmed motors never turn, whatever the throttle.
BEC / UBEC	a small regulator that turns battery voltage into 5 V for the electronics. UBEC = a separate one.
ALT HOLD	altitude hold: the drone keeps its height by itself; the throttle stick asks for climb or descent
Barometer	pressure sensor (BMP280): air pressure drops about 12 Pa per metre of height
Complementary filter	mixes a fast sensor that drifts with a slow sensor that doesn't, keeping the best of both
ESC	Electronic Speed Controller: drives one brushless motor from the signal it receives
Failsafe	what happens when something goes wrong (radio lost, battery empty). Here: in the air the drone lands by itself; on the ground the motors stop.
Gyro / accelerometer	the two sensors in the MPU6050: rotation speed, and direction of gravity
i-BUS	FlySky's protocol that sends all channels down one wire
Hover throttle	the throttle that holds the drone in the air; learned by the flight code while you fly
KV	motor speed per volt with no load: 2200KV × 12 V ≈ 26,000 rpm
Landing detector	the code that decides "we are on the ground" (asks for less throttle, nothing moves)
PDB	Power Distribution Board: splits the battery power to the 4 ESCs
PID	the controller that turns an error into a correction: Proportional, Integral, Derivative
PSRAM / flash	extra memory of the N16R8: 8 MB PSRAM (fast, lost at power-off: the log during flight) and 16 MB flash (kept: the program and the saved logs)
Roll / pitch / yaw	tilting sideways / tilting nose up-down / turning left-right
Sag	the battery voltage drop under load; it comes back when the load stops
Simulator (SITL)	software in the loop: the real flight code runs on a PC with a virtual drone

6.6 Resources: books, papers, courses, documentation

Full list with links and notes: `docs/resources.md`.

Resource	Why
Quan Quan, *Introduction to Multicopter Design and Control*, Springer, 2017 (doi.org/10.1007/978-981-10-3382-7)	the textbook closest to this project: design, sensors, estimation, control, failsafes
R. Mahony, V. Kumar, P. Corke, "Multirotor Aerial Vehicles: Modeling, Estimation, and Control of Quadrotor", IEEE Robotics & Automation Magazine, 2012	the standard tutorial on quadrotor dynamics and control
R. W. Beard, T. W. McLain, *Small Unmanned Aircraft: Theory and Practice*, Princeton University Press, 2012	sensors, state estimation, autopilot design
K. J. Åström, R. M. Murray, *Feedback Systems*, 2nd ed., Princeton, 2021 (free PDF on cds.caltech.edu)	PID and feedback explained
T. Kilby, B. Kilby, *Make: Getting Started with Drones*, Maker Media, 2015	beginner build book
S. Madgwick (2010) and R. Mahony et al. (IEEE TAC, 2008): orientation filters; S. Colton, "The Balance Filter" (MIT)	beyond our complementary filter
V. Kumar, *Robotics: Aerial Robotics*, University of Pennsylvania, Coursera	free-to-audit course
Brian Douglas, MATLAB Tech Talks: "Understanding PID Control", "Drone Simulation and Control"	short, clear videos
B. Beauregard, "Improving the Beginner's PID" (brettbeauregard.com)	PID windup, derivative kick
J. Brokking, YMFC-AL (brokking.net)	our loop structure and PID style
ArduPilot docs: Altitude Hold, Land, Battery and Radio failsafe (ardupilot.org/copter); PX4 safety and land detector (docs.px4.io); INAV navigation (github.com/iNavFlight/inav)	how mature autopilots do the same things
ESP-FC (github.com/rtlopez/esp-fc), Espressif ESP-Drone, N. Rehm's dRehmFlight	other open flight controllers on small boards
Bosch BMP280 datasheet, InvenSense MPU-6050 product specification and register map, Espressif ESP32-S3 Technical Reference Manual	the chips we drive directly
Battery University, BU-808 "How to Prolong Lithium-based Batteries"	LiPo care

DRONE RULES IN ALGERIA

Presidential Decree 21-285 (2021) and an interministerial decree of 8 September 2025 regulate owning, building and flying drones. In 2026 the Ministry of National Defence asked all owners to register their drones with the National Centre for Unmanned Aircraft Systems (CNSAPB). Ask the competition organisers how competitors' drones are authorised and fly only where you are allowed. Sources in `docs/resources.md`; this is not legal advice.

Appendix — Source code

The four sketches as in the project folder `esp32-drone-fc/firmware/`, printed for reference. Upload from the files; don't retype them. The bench test, the Python log tool and the simulator are in the project folder too.

A. 1_motor_test.ino

```
/*
  ============================================================
   SKETCH 1 — MOTOR TEST + ESC CALIBRATION   (version 2)
   ESP32-S3 flight controller board (your PCB)
  ============================================================
   Serial Monitor: 115200 baud, line ending "Newline".

   What changed in version 2 (fix for "ESCs keep beeping, motors don't spin"):
     - The ESC signal now starts at 50 Hz, the standard servo signal that
       every ESC accepts. Use  f  to try faster signals later.
     - Calibration is timed by the code: you only plug in the LiPo at "NOW".
     - New tools:  b = what the beeps mean,  p = wiring check with a multimeter,
                   i = status,  o = lower the "off" pulse.
     - Stronger output drive on the 4 signal pins.

   Commands (type, then Enter):
     c      ESC calibration (all 4 ESCs at once)          -- PROPS OFF
     1..4   spin ONE motor (M1..M4) for 3 seconds
     a      spin ALL 4 motors together for 3 seconds
     s      spin M1 -> M2 -> M3 -> M4, one after another
     +  -   raise / lower the test throttle (steps of 25 us)
     f      change the ESC signal speed: 50 / 100 / 200 / 250 / 400 Hz
     o      change the "off" pulse: 1000 / 950 / 900 us
     p      wiring check (multimeter) for the 4 signal wires
     b      what the ESC beeps mean
     i      status (signal, battery)
     h      show the menu

   EMERGENCY STOP (any of these, at any time):
     - type ANY key + Enter in the Serial Monitor
     - press the BOOT button on the ESP32 board
     - unplug the LiPo (always works, even if the code is stuck)
   Motors also stop by themselves after 3 seconds.

   Motor layout (seen from above, front at the top):
        M4 (CW)    M2 (CCW)
             \  /
             /  \
        M3 (CCW)   M1 (CW)
*/

#include <Arduino.h>
#include "driver/gpio.h"

// ---------------- Pins (from your PCB) ----------------
const uint8_t MOTOR_PINS[4] = {10, 11, 12, 13};      // H6 pins 1..4
const char* const MOTOR_NAMES[4] = {"M1 rear-right  (should spin CW)",
                                    "M2 front-right (should spin CCW)",
                                    "M3 rear-left   (should spin CCW)",
                                    "M4 front-left  (should spin CW)"};
const int BOOT_BUTTON = 0;                           // BOOT button on the DevKit
const int VBAT_PIN    = 1;                           // H5 battery divider (R3 10k / R4 3.3k)
const float VBAT_DIVIDER = (10000.0f + 3300.0f) / 3300.0f;

// ---------------- ESC signal ----------------
const uint32_t FREQ_LIST[]  = {50, 100, 200, 250, 400};   // f cycles through these
const uint8_t  FREQ_COUNT   = sizeof(FREQ_LIST) / sizeof(FREQ_LIST[0]);
uint8_t        freqIndex    = 0;                          // start at 50 Hz
```

```
uint32_t       escFreq      = 50;
const uint8_t  ESC_RES_BITS = 14;

const uint16_t OFF_LIST[]   = {1000, 950, 900};           // o cycles through these
const uint8_t  OFF_COUNT    = sizeof(OFF_LIST) / sizeof(OFF_LIST[0]);
uint8_t        offIndex     = 0;
uint16_t       pwmOff       = 1000;                       // "motor off" pulse
const uint16_t CAL_MIN      = 1000;                       // calibration end points
const uint16_t CAL_MAX      = 2000;

// ---------------- Test limits ----------------
const uint16_t TEST_MIN = 1050;
const uint16_t TEST_MAX = 1400;      // hard cap for bench tests
uint16_t testThrottle   = 1200;      // start low
const uint32_t RUN_TIME_MS = 3000;   // auto-stop after this

bool attachOk[4]     = {false, false, false, false};
bool bootStopEnabled = true;         // switched off if the BOOT pin reads "pressed" at start-up
bool lipoSeen        = false;

// ============================================================
//  ESC output
// ============================================================
uint32_t usToDuty(uint16_t us, uint32_t freq) {
  const uint32_t periodUs = 1000000UL / freq;
  const uint32_t maxDuty  = (1UL << ESC_RES_BITS) - 1;
  return (uint32_t)us * maxDuty / periodUs;
}

void escWriteDuty(uint8_t m, uint32_t duty) {
#if ESP_ARDUINO_VERSION_MAJOR >= 3
  ledcWrite(MOTOR_PINS[m], duty);
#else
  ledcWrite(m, duty);
#endif
}

void escWriteUs(uint8_t m, uint16_t us) {
  escWriteDuty(m, usToDuty(us, escFreq));
}

void allMotors(uint16_t us) {
  for (uint8_t m = 0; m < 4; m++) escWriteUs(m, us);
}

void allNoSignal() {                  // wire held LOW: the ESC sees "no signal" (motor stops)
  for (uint8_t m = 0; m < 4; m++) escWriteDuty(m, 0);
}

void escInit() {
  for (uint8_t m = 0; m < 4; m++) {
#if ESP_ARDUINO_VERSION_MAJOR >= 3
    attachOk[m] = ledcAttach(MOTOR_PINS[m], escFreq, ESC_RES_BITS);
#else
    attachOk[m] = ledcSetup(m, escFreq, ESC_RES_BITS) > 0;
    ledcAttachPin(MOTOR_PINS[m], m);
#endif
    gpio_set_drive_capability((gpio_num_t)MOTOR_PINS[m], GPIO_DRIVE_CAP_3);   // strongest output
  }
  allMotors(pwmOff);
}

// Change the signal speed safely: no pulses during the switch, so an ESC
// can never see an over-long pulse (which some read as full throttle).
bool setEscFrequency(uint32_t f) {
  allNoSignal();
  delay(30);
  bool ok = true;
  for (uint8_t m = 0; m < 4; m++) {
#if ESP_ARDUINO_VERSION_MAJOR >= 3
    if (ledcChangeFrequency(MOTOR_PINS[m], f, ESC_RES_BITS) == 0) ok = false;
#else
```

```
    if (ledcSetup(m, f, ESC_RES_BITS) == 0) ok = false;
#endif
  }
  escFreq = f;
  allMotors(pwmOff);
  return ok;
}

// ============================================================
//  Battery (only works if H5 is wired to the PDB; reads 0 V otherwise)
// ============================================================
float readVbat() {
  uint32_t mv = 0;
  for (int i = 0; i < 8; i++) mv += analogReadMilliVolts(VBAT_PIN);
  return (mv / 8) / 1000.0f * VBAT_DIVIDER;
}

void lipoWatch() {
  static uint32_t last = 0;
  if (millis() - last < 300) return;
  last = millis();
  float v = readVbat();
  if (!lipoSeen && v > 6.0f) {
    lipoSeen = true;
    Serial.printf("\nLiPo connected (%.1f V). Listen: melody, 3 beeps, long beep = ESCs READY.\n", v);
    Serial.println(F("  Beeping that doesn't stop? Type b."));
  } else if (lipoSeen && v < 3.0f) {
    lipoSeen = false;
    Serial.println(F("\nLiPo disconnected."));
  }
}

// ============================================================
//  Stop handling / keyboard
// ============================================================
void flushSerial() {
  delay(20);
  while (Serial.available()) Serial.read();
}

bool bootPressed() {
  return bootStopEnabled && digitalRead(BOOT_BUTTON) == LOW;
}

// true if the user asked to stop (Serial key or BOOT button)
bool stopRequested() {
  if (Serial.available()) { flushSerial(); return true; }
  return bootPressed();
}

void emergencyStop() {
  allMotors(pwmOff);
  Serial.println(F("\n*** STOPPED: all motors off ***"));
  while (bootPressed()) delay(10);   // wait for button release
}

// Wait for a key (returns it), the BOOT button (returns 'B'),
// or a timeout (returns 0). timeoutMs = 0 means wait forever.
char waitKeyOrBoot(uint32_t timeoutMs) {
  flushSerial();
  while (bootPressed()) delay(10);   // ignore a button that is already held
  uint32_t t0 = millis();
  while (timeoutMs == 0 || millis() - t0 < timeoutMs) {
    if (Serial.available()) { char c = Serial.read(); flushSerial(); return c; }
    if (bootPressed()) { while (bootPressed()) delay(10); return 'B'; }
    delay(5);
  }
  return 0;
}

// ============================================================
//  Spin the motors in 'mask' (bit 0 = M1 ... bit 3 = M4)
```

```
//  Soft ramp up, hold, then stop. Returns false if stopped early.
// ============================================================
bool runMotors(uint8_t mask, uint32_t holdMs) {
  for (uint16_t us = pwmOff; us <= testThrottle; us += 5) {
    for (uint8_t m = 0; m < 4; m++) if (mask & (1 << m)) escWriteUs(m, us);
    delay(10);
    if (stopRequested()) { emergencyStop(); return false; }
  }
  uint32_t t0 = millis();
  while (millis() - t0 < holdMs) {
    if (stopRequested()) { emergencyStop(); return false; }
    delay(5);
  }
  allMotors(pwmOff);
  return true;
}

void notSpinningHint() {
  Serial.println(F("  Nothing turned? If the ESCs are still beeping, they are NOT armed:"));
  Serial.println(F("  type b to read the beeps. If they are silent, try + to raise the throttle."));
}

void spinOne(uint8_t m) {
  Serial.printf("\n%s  ->  %u us for %lu s at %lu Hz   (any key / BOOT = STOP)\n",
                MOTOR_NAMES[m], testThrottle, (unsigned long)(RUN_TIME_MS / 1000),
                (unsigned long)escFreq);
  if (runMotors(1 << m, RUN_TIME_MS)) {
    Serial.println(F("  done. Which arm spun? Which direction (from above)?"));
    notSpinningHint();
  }
}

void spinAll() {
  Serial.printf("\nALL 4 motors  ->  %u us for %lu s at %lu Hz   (any key / BOOT = STOP)\n",
                testThrottle, (unsigned long)(RUN_TIME_MS / 1000), (unsigned long)escFreq);
  if (runMotors(0x0F, RUN_TIME_MS)) {
    Serial.println(F("  done. Did all 4 start at the same time and sound the same?"));
    notSpinningHint();
  }
}

void spinSequence() {
  for (uint8_t m = 0; m < 4; m++) {
    Serial.printf("\n%s  ->  %u us\n", MOTOR_NAMES[m], testThrottle);
    if (!runMotors(1 << m, 2000)) return;
    uint32_t t0 = millis();
    while (millis() - t0 < 1000) {                    // pause, still listening
      if (stopRequested()) { emergencyStop(); return; }
      delay(5);
    }
  }
  Serial.println(F("\nSequence done."));
}

// ============================================================
//  ESC calibration: teaches each ESC where 1000 us and 2000 us are.
//  The code does the timing: you plug in the LiPo at "NOW", and it drops
//  to STOP 3-4 s later (or as soon as you press Enter / BOOT on 'beep-beep').
// ============================================================
void calibrateEscs() {
  Serial.println(F("\n========== ESC CALIBRATION =========="));
  Serial.println(F("Teaches every ESC: 1000 us = stop, 2000 us = full power. PROPS OFF!"));
  if (readVbat() > 6.0f) {
    Serial.println(F("The LiPo is plugged in (H5 sees it). UNPLUG it, then type c again."));
    return;
  }
  Serial.println(F("1) LiPo UNPLUGGED? Hold the LiPo plug ready. Press Enter to start. (q = cancel)"));
  if (waitKeyOrBoot(0) == 'q') { Serial.println(F("Cancelled.")); return; }

  allMotors(CAL_MAX);
  Serial.println(F("2) Signal is now FULL (2000 us). Plug in the LiPo when you read NOW:"));
  for (int i = 3; i >= 1; i--) {
```

```
    Serial.printf("   %d...\n", i);
    uint32_t t0 = millis();
    while (millis() - t0 < 1000) {
      if (Serial.available() && Serial.read() == 'q') {
        allMotors(pwmOff);
        flushSerial();
        Serial.println(F("Cancelled."));
        return;
      }
      delay(5);
    }
  }
  Serial.println(F("   >>> NOW: PLUG IN THE LIPO <<<"));
  flushSerial();

  // Drop to STOP: 3 s after H5 sees the LiPo, or 4 s after NOW if H5 isn't wired.
  // Enter or BOOT drops it immediately (use it when you hear 'beep-beep').
  uint32_t tGo = millis(), tPlug = 0;
  while (true) {
    uint32_t now = millis();
    if (tPlug == 0 && readVbat() > 6.0f) { tPlug = now; Serial.println(F("   LiPo detected.")); }
    if (tPlug != 0 && now - tPlug >= 3000) break;
    if (tPlug == 0 && now - tGo >= 4000) break;
    if (Serial.available() || bootPressed()) break;
    delay(10);
  }
  allMotors(CAL_MIN);                  // drop to STOP right away...
  flushSerial();                       // ...then tidy up the key / button
  while (bootPressed()) delay(10);

  Serial.println(F("3) Signal is now STOP (1000 us). What did you hear?"));
  Serial.println(F("   'beep-beep', then 3 short beeps + 1 long beep  -> calibration SAVED."));
  Serial.println(F("   only the normal start-up tones                 -> the ESC missed FULL:"));
  Serial.println(F("        type c again and plug in exactly at NOW."));
  Serial.println(F("   a melody, then beep patterns that repeat     -> PROGRAMMING MODE:"));
  Serial.println(F("        unplug the LiPo, type c again, plug in a little LATER after NOW."));
  Serial.println(F("4) Unplug the LiPo, wait 5 s, plug it back in, listen for: melody, 3 beeps, long
beep."));
}

// ============================================================
//  Wiring check: one signal wire at a time gets a SAFE "motor off"
//  pulse (1000 us at 250 Hz); the others get nothing. A multimeter on
//  DC volts reads about 0.8 V on the active wire and 0 V on the others.
// ============================================================
void pinCheck() {
  Serial.println(F("\n=========== WIRING CHECK (multimeter) ==========="));
  Serial.println(F("Best with the LiPo UNPLUGGED. Multimeter on DC volts (2 V or 20 V range)."));
  Serial.println(F("Black probe on H6 pin 5 (GND). Red probe on the pin named below."));
  Serial.println(F("Expect about 0.8 V on that pin, and 0 V on the other three."));
  Serial.println(F("Then measure at that ESC's plug too: white/yellow wire vs black/brown wire."));
  Serial.println(F("Enter = next wire, q = quit."));

  const uint32_t savedFreq = escFreq;
  setEscFrequency(250);
  for (uint8_t m = 0; m < 4; m++) {
    allNoSignal();
    escWriteUs(m, 1000);
    Serial.printf("\n  NOW ACTIVE: M%u  (GPIO %u, H6 pin %u)  -> expect ~0.8 V on H6 pin %u only\n",
                  m + 1, MOTOR_PINS[m], m + 1, m + 1);
    if (waitKeyOrBoot(0) == 'q') break;
  }
  setEscFrequency(savedFreq);
  Serial.println(F("\nWiring check finished. Signals are back to normal."));
  Serial.println(F("  0 V on the active pin  -> broken trace/solder joint between the ESP32 and H6."));
  Serial.println(F("  0.8 V on the wrong pin -> the plug or the wires are in the wrong order."));
  Serial.println(F("  Right at H6 but 0 V at the ESC plug -> plug reversed or bad crimp."));
  Serial.println(F("  Also check (LiPo unplugged, beep mode): H6 pin 5 <-> PDB minus must BEEP."));
}

// ============================================================
//  Beep decoder (Hobbywing / XXD / SimonK-style ESCs)
```

```
// ============================================================
void printBeeps() {
  Serial.println(F("\n=========== WHAT THE ESC BEEPS MEAN ==========="));
  Serial.println(F("Melody, then 3 short beeps, then 1 long beep, then SILENCE"));
  Serial.println(F("   -> NORMAL. Armed and ready (3 beeps = 3S battery). Motors will spin."));
  Serial.println(F("One beep every ~2 seconds, forever"));
  Serial.println(F("   -> NO VALID SIGNAL. Check the ESC signal wire on H6 and ESC ground on H6 pin 5,"));
  Serial.println(F("      run p (wiring check), try another speed with f, then unplug/replug the LiPo."));
  Serial.println(F("Fast beeps (about 4 per second), forever"));
  Serial.println(F("   -> THROTTLE NOT AT ZERO: the ESC thinks 'off' is too high."));
  Serial.println(F("      Run c (calibration). Or lower the off pulse with o, then replug the LiPo."));
  Serial.println(F("'beep-beep' pairs, about one pair per second"));
  Serial.println(F("   -> BATTERY VOLTAGE WRONG (LiPo too low, or the ESC's battery setting changed)."));
  Serial.println(F("      Measure the LiPo: a charged 3S reads 11.1-12.6 V. Charge it."));
  Serial.println(F("'beep-beep', then a melody, then beep patterns that repeat"));
  Serial.println(F("   -> PROGRAMMING MODE (the ESC saw full throttle too long at power-up)."));
  Serial.println(F("      Unplug the LiPo. Nothing is spinning; just redo c if you were calibrating."));
  Serial.println(F("No sound at all"));
  Serial.println(F("   -> No power to the ESC: check its thick red/black wires on the PDB."));
}

// ============================================================
//  Menu / status
// ============================================================
void printMenu() {
  Serial.println(F("\n=========== SKETCH 1: MOTOR TEST (v2) ==========="));
  Serial.println(F("  !!! PROPS OFF (until guide Step 6) !!!"));
  Serial.println(F("  c     ESC calibration"));
  Serial.println(F("  1-4   spin one motor          a  all 4 together"));
  Serial.println(F("  s     M1 -> M4 in sequence    + -  test throttle"));
  Serial.println(F("  f     ESC signal speed        o  off pulse"));
  Serial.println(F("  p     wiring check            b  what the beeps mean"));
  Serial.println(F("  i     status                  h  this menu"));
  Serial.println(F("  STOP: any key, BOOT button, or unplug the LiPo"));
}

void printStatus() {
  Serial.println(F("\n--- STATUS ---"));
  Serial.printf("ESC signal    : %lu Hz, off pulse %u us, test throttle %u us (max %u)\n",
                (unsigned long)escFreq, pwmOff, testThrottle, TEST_MAX);
  for (uint8_t m = 0; m < 4; m++)
    Serial.printf("  M%u -> GPIO %u : %s\n", m + 1, MOTOR_PINS[m],
                  attachOk[m] ? "signal running" : "LEDC SETUP FAILED (report this)");
  float v = readVbat();
  if (v > 6.0f) Serial.printf("LiPo (via H5) : %.2f V  (%.2f V per cell)\n", v, v / 3.0f);
  else          Serial.println(F("LiPo (via H5) : not seen (unplugged, or H5 not wired - that's OK)"));
  if (!bootStopEnabled) Serial.println(F("BOOT button   : reads 'pressed' at start-up -> BOOT stop
disabled"));
}

void setup() {
  escInit();                            // FIRST: ESCs see "motor off" immediately
  pinMode(BOOT_BUTTON, INPUT_PULLUP);
  analogReadResolution(12);
  Serial.begin(115200);
  delay(1500);
  bootStopEnabled = (digitalRead(BOOT_BUTTON) == HIGH);
  printMenu();
  printStatus();
  Serial.println(F("\nNow plug in the LiPo and LISTEN."));
  Serial.println(F("Normal = melody, 3 beeps, 1 long beep, then silence. Then type 1."));
  Serial.println(F("Beeping that never stops? Type b."));
}

void loop() {
  allMotors(pwmOff);
  lipoWatch();
  if (!Serial.available()) { delay(5); return; }

  char c = Serial.read();
  if (c == '\n' || c == '\r' || c == ' ') return;
  flushSerial();
```

```
  switch (c) {
    case 'c': calibrateEscs(); break;
    case '1': case '2': case '3': case '4': spinOne(c - '1'); break;
    case 'a': spinAll(); break;
    case 's': spinSequence(); break;
    case '+':
      testThrottle += 25;
      if (testThrottle > TEST_MAX) testThrottle = TEST_MAX;
      Serial.printf("test throttle = %u us\n", testThrottle);
      break;
    case '-':
      testThrottle -= 25;
      if (testThrottle < TEST_MIN) testThrottle = TEST_MIN;
      Serial.printf("test throttle = %u us\n", testThrottle);
      break;
    case 'f': {
      freqIndex = (freqIndex + 1) % FREQ_COUNT;
      bool ok = setEscFrequency(FREQ_LIST[freqIndex]);
      Serial.printf("ESC signal = %lu Hz%s\n", (unsigned long)escFreq, ok ? "" : "  (LEDC ERROR)");
      Serial.println(F("  Unplug the LiPo, plug it back in, listen, then test with 1-4."));
      Serial.println(F("  Note the FASTEST speed that arms and spins smoothly: sketch 4 needs it."));
      break;
    }
    case 'o':
      offIndex = (offIndex + 1) % OFF_COUNT;
      pwmOff = OFF_LIST[offIndex];
      allMotors(pwmOff);
      Serial.printf("off pulse = %u us. Unplug and replug the LiPo, then listen.\n", pwmOff);
      break;
    case 'p': pinCheck(); break;
    case 'b': printBeeps(); break;
    case 'i': printStatus(); break;
    case 'h': printMenu(); break;
    default:  Serial.println(F("? (h = menu)")); break;
  }
  allMotors(pwmOff);
}
```

B. 2_receiver_test.ino

```
/*
  ================================================================
   SKETCH 2 — RECEIVER TEST (FlySky FS-iA10B, i-BUS)
  ================================================================
   Serial Monitor: 115200 baud.

   Wiring: receiver "i-BUS SERVO" port -> PCB H2 (CONN_RC)
       S (signal) -> H2 pin 2  (GPIO 18)
       +          -> H2 pin 1  (5V)
       -          -> H2 pin 3  (GND)

   What you should see, 10 times per second:
     ROLL PITCH THR YAW  = CH1..CH4, about 1000..2000, 1500 at centre
     CH5 (SwA)           = ARM (about 2000) or SAFE (about 1000)
     CH6 (SwC)           = MANUAL (about 1000), ALT HOLD (about 1500), LAND (about 2000)
     CH7..CH10           = other switches / knobs (not used)
   Failsafe check (guide Step 7.6): with SwA on ARM and SwC on MANUAL, switch the
   transmitter OFF. Within about 1 s you must see THR about 1000, CH5 still ARM
   and CH6 = LAND  (the flight controller then lands by itself).
   Motors are held at "off" the whole time (safe to run with the LiPo in).
   The ESCs get a 50 Hz "off" signal here, so they arm and stay quiet.
*/

#include <Arduino.h>

const int IBUS_RX_PIN = 18;
const uint8_t MOTOR_PINS[4] = {10, 11, 12, 13};

// ---------------- i-BUS ----------------
const uint8_t IBUS_CHANNELS = 14;
uint16_t rc[IBUS_CHANNELS];
uint32_t lastFrameMs = 0;
uint32_t goodFrames = 0, badFrames = 0;

// i-BUS frame: 0x20 0x40, 14 channels (2 bytes each, little-endian), checksum
void ibusUpdate() {
  static uint8_t buf[32];
  static uint8_t idx = 0;
  while (Serial1.available()) {
    uint8_t b = Serial1.read();
    if (idx == 0) { if (b == 0x20) buf[idx++] = b; continue; }
    if (idx == 1) {
      if (b == 0x40) buf[idx++] = b;
      else if (b != 0x20) idx = 0;
      continue;
    }
    buf[idx++] = b;
    if (idx == 32) {
      idx = 0;
      uint16_t sum = 0xFFFF;
      for (uint8_t i = 0; i < 30; i++) sum -= buf[i];
      if (sum == (uint16_t)(buf[30] | (buf[31] << 8))) {
        for (uint8_t ch = 0; ch < IBUS_CHANNELS; ch++)
          rc[ch] = (buf[2 + ch * 2] | (buf[3 + ch * 2] << 8)) & 0x0FFF;
        lastFrameMs = millis();
        goodFrames++;
      } else {
        badFrames++;
      }
    }
  }
}

// Keep the ESCs quiet and safe: 1000 us at 50 Hz (a signal every ESC accepts)
void escHoldOff() {
  const uint32_t duty = 1000UL * ((1UL << 14) - 1) / 20000UL;  // 1000 us of a 20000 us (50 Hz) period
  for (uint8_t m = 0; m < 4; m++) {
#if ESP_ARDUINO_VERSION_MAJOR >= 3
    ledcAttach(MOTOR_PINS[m], 50, 14);
```

```
    ledcWrite(MOTOR_PINS[m], duty);
#else
    ledcSetup(m, 50, 14);
    ledcAttachPin(MOTOR_PINS[m], m);
    ledcWrite(m, duty);
#endif
  }
}

// Same thresholds as the flight controller (sketch 4)
const char* armSwitch(uint16_t v) { return v > 1700 ? "ARM" : "SAFE"; }
const char* modeSwitch(uint16_t v) {
  if (v < 1300) return "MANUAL";
  if (v > 1700) return "LAND";
  return "ALT HOLD";
}

void setup() {
  escHoldOff();
  Serial.begin(115200);
  Serial1.begin(115200, SERIAL_8N1, IBUS_RX_PIN, -1);
  delay(1500);
  Serial.println(F("\n=========== SKETCH 2: RECEIVER TEST ==========="));
  Serial.println(F("Turn the transmitter ON. Move each stick and switch."));
  Serial.println(F("CH5 must show ARM/SAFE with SwA, CH6 MANUAL/ALT HOLD/LAND with SwC."));
}

void loop() {
  ibusUpdate();

  static uint32_t lastPrint = 0;
  if (millis() - lastPrint < 100) return;
  lastPrint = millis();

  if (lastFrameMs == 0 || millis() - lastFrameMs > 200) {
    Serial.printf("SIGNAL LOST   (good frames %lu, bad frames %lu)\n",
                  (unsigned long)goodFrames, (unsigned long)badFrames);
    return;
  }

  Serial.printf("ROLL %4u  PITCH %4u  THR %4u  YAW %4u | "
                "CH5 %4u %-4s | CH6 %4u %-8s | CH7 %4u  CH8 %4u  CH9 %4u  CH10 %4u\n",
                rc[0], rc[1], rc[2], rc[3],
                rc[4], armSwitch(rc[4]), rc[5], modeSwitch(rc[5]), rc[6], rc[7], rc[8], rc[9]);
}
```

C. 3_imu_test.ino

```
/*
  ================================================================
   SKETCH 3 — MPU6050 TEST: orientation + level calibration
  ================================================================
   Serial Monitor: 115200 baud, line ending "Newline".

   The MPU6050 (GY-521) plugs into U2 pins 1..8 (VCC GND SCL SDA XDA XCL AD0 INT).
   Pins 9-10 of U2 stay empty. AD0 is tied to GND -> I2C address 0x68.

   Commands:
     g   re-measure gyro offsets (keep the drone perfectly still)
     l   level calibration (drone on a LEVEL surface) -> prints the 2 trim values
     h   help / checklist

   The 3 checks (front of the drone = where M2/M4 are):
     1. Tilt the RIGHT side down   -> ROLL  becomes POSITIVE (+)
     2. Lift the NOSE up           -> PITCH becomes POSITIVE (+)
     3. Turn the nose to the RIGHT -> YAW RATE is POSITIVE (+) while turning
   If one is wrong, change IMU_ROTATION / IMU_UPSIDE_DOWN below, upload, test again.
   When all 3 pass, copy the SAME settings into sketch 4's config.h (section 1).
*/

#include <Arduino.h>
#include <Wire.h>
#include "esp_log.h"

// ===================== SETTINGS TO FIND =====================
// Direction the "X" arrow printed on the GY-521 points, seen from above:
//   0 = forward, 90 = right, 180 = backward, 270 = left
const int  IMU_ROTATION    = 0;
const bool IMU_UPSIDE_DOWN = false;   // true if the chip faces the ground
// ============================================================

const int I2C_SDA_PIN = 9;
const int I2C_SCL_PIN = 8;
const uint8_t MOTOR_PINS[4] = {10, 11, 12, 13};
const uint8_t MPU_ADDR = 0x68;

const float GYRO_LSB = 65.5f;     // +-500 deg/s range
const float RAD2DEG  = 57.29578f;
const float DEG2RAD  = 0.0174533f;
const uint32_t LOOP_US = 4000;    // 250 Hz, same as the flight code

float gyroOff[3] = {0, 0, 0};
float rollTrim = 0, pitchTrim = 0;
float angleRoll = 0, anglePitch = 0;
float rollRate = 0, pitchRate = 0, yawRate = 0;
float accRoll = 0, accPitch = 0;

// ---------------- I2C helpers ----------------
int readReg(uint8_t reg) {
  Wire.beginTransmission(MPU_ADDR);
  Wire.write(reg);
  if (Wire.endTransmission(false) != 0) return -1;
  if (Wire.requestFrom(MPU_ADDR, (uint8_t)1) != 1) return -1;
  return Wire.read();
}

bool writeReg(uint8_t reg, uint8_t val) {
  Wire.beginTransmission(MPU_ADDR);
  Wire.write(reg);
  Wire.write(val);
  return Wire.endTransmission() == 0;
}

bool readRaw(int16_t r[6]) {             // ax ay az gx gy gz
  Wire.beginTransmission(MPU_ADDR);
  Wire.write(0x3B);
  if (Wire.endTransmission(false) != 0) return false;
```

```
  if (Wire.requestFrom(MPU_ADDR, (uint8_t)14) != 14) return false;
  uint8_t b[14];
  for (int i = 0; i < 14; i++) b[i] = Wire.read();
  r[0] = (int16_t)(b[0] << 8 | b[1]);
  r[1] = (int16_t)(b[2] << 8 | b[3]);
  r[2] = (int16_t)(b[4] << 8 | b[5]);
  r[3] = (int16_t)(b[8] << 8 | b[9]);
  r[4] = (int16_t)(b[10] << 8 | b[11]);
  r[5] = (int16_t)(b[12] << 8 | b[13]);
  return true;
}

bool mpuInit() {
  int who = readReg(0x75);
  if (who < 0) return false;
  Serial.printf("MPU found, WHO_AM_I = 0x%02X %s\n", who,
                who == 0x68 ? "(MPU6050, OK)" : "(not 0x68: clone chip, usually still works)");
  writeReg(0x6B, 0x01);   // wake up, clock = gyro PLL
  delay(50);
  writeReg(0x1A, 0x03);   // digital low-pass filter ~44 Hz (cuts motor vibration)
  writeReg(0x1B, 0x08);   // gyro  +-500 deg/s
  writeReg(0x1C, 0x10);   // accel +-8 g
  return true;
}

// Turn chip axes into drone axes: forward / left / up
void mapAxes(float x, float y, float z, float &fwd, float &left, float &up) {
  switch (IMU_ROTATION) {
    case 90:  fwd =  y; left = -x; break;
    case 180: fwd = -x; left = -y; break;
    case 270: fwd = -y; left =  x; break;
    default:  fwd =  x; left =  y; break;
  }
  up = z;
  if (IMU_UPSIDE_DOWN) { left = -left; up = -up; }
}

// Read the sensor and compute rates (deg/s) and accel angles (deg)
bool updateImu() {
  int16_t r[6];
  if (!readRaw(r)) return false;
  float gf, gl, gu, af, al, au;
  mapAxes((r[3] - gyroOff[0]) / GYRO_LSB, (r[4] - gyroOff[1]) / GYRO_LSB,
          (r[5] - gyroOff[2]) / GYRO_LSB, gf, gl, gu);
  rollRate  =  gf;     // + = right side going down
  pitchRate = -gl;     // + = nose going up
  yawRate   = -gu;     // + = nose turning right
  mapAxes(r[0], r[1], r[2], af, al, au);
  accRoll  = atan2f(al, au) * RAD2DEG - rollTrim;
  accPitch = atan2f(af, sqrtf(al * al + au * au)) * RAD2DEG - pitchTrim;
  return true;
}

void calibrateGyro() {
  Serial.println(F("Gyro calibration: keep the drone STILL for 4 seconds..."));
  double sum[3] = {0, 0, 0};
  int n = 0;
  int16_t r[6];
  for (int i = 0; i < 1000; i++) {
    if (readRaw(r)) { sum[0] += r[3]; sum[1] += r[4]; sum[2] += r[5]; n++; }
    delay(4);
  }
  if (n == 0) { Serial.println(F("  FAILED: no data from MPU")); return; }
  for (int k = 0; k < 3; k++) gyroOff[k] = sum[k] / n;
  Serial.printf("  done. Raw gyro offsets: %.0f %.0f %.0f\n", gyroOff[0], gyroOff[1], gyroOff[2]);
  updateImu();
  angleRoll = accRoll;
  anglePitch = accPitch;
}

void levelCalibration() {
  Serial.println(F("Level calibration: drone on a LEVEL surface, don't touch it (3 s)..."));
```

```
  float oldR = rollTrim, oldP = pitchTrim;
  rollTrim = pitchTrim = 0;
  double sr = 0, sp = 0;
  int n = 0;
  for (int i = 0; i < 750; i++) {
    if (updateImu()) { sr += accRoll; sp += accPitch; n++; }
    delay(4);
  }
  if (n == 0) { rollTrim = oldR; pitchTrim = oldP; Serial.println(F("  FAILED")); return; }
  rollTrim = sr / n;
  pitchTrim = sp / n;
  Serial.println(F("  done. In sketch 4's config.h, REPLACE the two ACC_..._TRIM lines with:"));
  Serial.printf("  #define ACC_ROLL_TRIM     %.2ff\n", rollTrim);
  Serial.printf("  #define ACC_PITCH_TRIM    %.2ff\n", pitchTrim);
  angleRoll = 0;
  anglePitch = 0;
}

void printHelp() {
  Serial.println(F("\n=========== SKETCH 3: MPU6050 TEST ==========="));
  Serial.printf("Settings: IMU_ROTATION %d, IMU_UPSIDE_DOWN %s  (same in sketch 4's config.h)\n",
                IMU_ROTATION, IMU_UPSIDE_DOWN ? "true" : "false");
  Serial.println(F("Checks (front = M2/M4 side):"));
  Serial.println(F("  1. Right side DOWN  -> ROLL  positive"));
  Serial.println(F("  2. Nose UP          -> PITCH positive"));
  Serial.println(F("  3. Turn nose RIGHT  -> YAW RATE positive (while turning)"));
  Serial.println(F("Commands: g = gyro calibration, l = level calibration, h = help"));
}

// ============================================================
//  I2C check: runs when the MPU6050 doesn't answer.
//  Uses the idle bus voltages measured at start-up + a scan of every address.
// ============================================================
const char* lineState(float v) {
  if (v > 2.6f) return "OK (pulled up)";
  if (v < 0.3f) return "STUCK LOW";
  return "HALF-WAY (pulled down by something)";
}

void i2cDoctor(float sdaV, float sclV) {
  Serial.println(F("\n=========== MPU6050 NOT FOUND: I2C CHECK ==========="));
  Serial.println(F("Bus voltage at start-up (both should be about 3.1-3.3 V):"));
  Serial.printf("  SDA (GPIO %d, U2 pin 4): %.2f V  %s\n", I2C_SDA_PIN, sdaV, lineState(sdaV));
  Serial.printf("  SCL (GPIO %d, U2 pin 3): %.2f V  %s\n", I2C_SCL_PIN, sclV, lineState(sclV));

  Serial.println(F("Scanning every I2C address..."));
  uint8_t found = 0;
  bool at69 = false;
  for (uint8_t a = 1; a < 127; a++) {
    Wire.beginTransmission(a);
    if (Wire.endTransmission() == 0) {
      found++;
      Serial.printf("  device at 0x%02X", a);
      if (a == 0x68)                Serial.print(F("  <- MPU6050, but it didn't answer a read: loose
contact?"));
      else if (a == 0x69)          { Serial.print(F("  <- MPU6050 with AD0 HIGH")); at69 = true; }
      else if (a == 0x76 || a == 0x77) Serial.print(F("  <- BMP280 barometer"));
      Serial.println();
    }
  }
  if (!found) Serial.println(F("  nothing answers on the bus."));

  Serial.print(F("\nMOST LIKELY: "));
  if (sdaV < 0.3f || sclV < 0.3f)
    Serial.println(F("SDA or SCL is shorted to GND (solder bridge), or the module sits in the wrong
holes."));
  else if (sdaV <= 2.6f || sclV <= 2.6f)
    Serial.println(F("the module has NO POWER (VCC/GND not connected) or is plugged in backwards."));
  else if (at69)
    Serial.println(F("AD0 is not tied to GND: check U2 pin 7 and its solder joint."));
  else if (!found)
    Serial.println(F("the module's pins don't touch the PCB (header not soldered) or the module is dead."));
```

```
  else
    Serial.println(F("a loose contact on SDA/SCL: re-solder U2 pins 3 and 4."));

  Serial.println(F("\nCheck, with the USB cable UNPLUGGED:"));
  Serial.println(F(" 1. The pin header is SOLDERED to the GY-521 board (it comes loose in the bag)."));
  Serial.println(F(" 2. Module in U2 pins 1-8, its VCC pin in the hole marked VCC; U2 pins 9-10 empty."));
  Serial.println(F(" 3. Beep test: module SDA <-> ESP32 pin 9, module SCL <-> ESP32 pin 8, GND <-> GND."));
  Serial.println(F("Then with USB plugged in:"));
  Serial.println(F(" 4. The small LED on the GY-521 is ON. No LED = no power."));
  Serial.println(F(" 5. Multimeter: module VCC to GND = about 3.3 V."));
  Serial.println(F(" 6. Still nothing? Connect the module straight to the ESP32 with 4 jumper wires"));
  Serial.println(F("    (VCC->3V3, GND->GND, SDA->9, SCL->8) and press RST."));
  Serial.println(F("    Found that way = the PCB is the problem. Still not found = the module is bad."));
  Serial.println(F("\nUnplug USB before touching wires. Press RST to test again after a fix."));
}

void setup() {
  // keep the ESCs at "off" while testing (50 Hz: every ESC accepts it)
  const uint32_t duty = 1000UL * ((1UL << 14) - 1) / 20000UL;  // 1000 us at 50 Hz
  for (uint8_t m = 0; m < 4; m++) {
#if ESP_ARDUINO_VERSION_MAJOR >= 3
    ledcAttach(MOTOR_PINS[m], 50, 14);
    ledcWrite(MOTOR_PINS[m], duty);
#else
    ledcSetup(m, 50, 14);
    ledcAttachPin(MOTOR_PINS[m], m);
    ledcWrite(m, duty);
#endif
  }

  Serial.begin(115200);
  delay(1500);

  // Hide the I2C driver's own red "nack" lines: this sketch explains problems itself.
  esp_log_level_set("i2c.master", ESP_LOG_NONE);
  esp_log_level_set("i2c", ESP_LOG_NONE);

  // Idle bus voltage, measured BEFORE the I2C driver takes the pins.
  pinMode(I2C_SDA_PIN, INPUT);
  pinMode(I2C_SCL_PIN, INPUT);
  delay(10);
  const float sdaV = analogReadMilliVolts(I2C_SDA_PIN) / 1000.0f;
  const float sclV = analogReadMilliVolts(I2C_SCL_PIN) / 1000.0f;

  Wire.begin(I2C_SDA_PIN, I2C_SCL_PIN, 400000);

  if (!mpuInit()) {
    i2cDoctor(sdaV, sclV);
    while (true) delay(1000);
  }
  printHelp();
  calibrateGyro();
}

void loop() {
  static uint32_t loopTimer = micros();
  while (micros() - loopTimer < LOOP_US) { }
  float dt = (micros() - loopTimer) / 1e6f;
  loopTimer = micros();
  if (dt > 0.02f) dt = 0.02f;

  if (updateImu()) {
    // Same filter as the flight code: gyro for fast moves, accel to stop drift
    angleRoll  += rollRate * dt;
    anglePitch += pitchRate * dt;
    float s = sinf(yawRate * dt * DEG2RAD);
    anglePitch -= angleRoll * s;
    angleRoll  += anglePitch * s;
    angleRoll  = 0.998f * angleRoll  + 0.002f * accRoll;
    anglePitch = 0.998f * anglePitch + 0.002f * accPitch;
  }
```

```
  if (Serial.available()) {
    char c = Serial.read();
    delay(20);
    while (Serial.available()) Serial.read();
    if (c == 'g') calibrateGyro();
    else if (c == 'l') levelCalibration();
    else if (c == 'h') printHelp();
    loopTimer = micros();
  }

  static uint32_t lastPrint = 0;
  if (millis() - lastPrint >= 150) {
    lastPrint = millis();
    Serial.printf("ROLL %+6.1f  PITCH %+6.1f deg  |  rates  roll %+7.1f  pitch %+7.1f  YAW %+7.1f deg/s\n",
                  angleRoll, anglePitch, rollRate, pitchRate, yawRate);
  }
}
```

D1. 4_flight_controller.ino

```
/*
  ============================================================
   SKETCH 4 (v3) - FLIGHT CONTROLLER
   ESP32-S3 + MPU6050 + BMP280 + FlySky i-BUS + 4 ESCs, Quad X
  ============================================================
   Flight modes (SwC = CH6, check with sketch 2):
     ~1000  MANUAL    self-levelling, you control the throttle
     ~1500  ALT HOLD  the drone keeps its height by itself (barometer).
                      Throttle stick in the middle = hold, up/down = climb/descend.
     ~2000  LAND      comes down slowly and stops the motors on the ground

   ARM   : SwC on MANUAL, throttle fully DOWN, then SwA (CH5) to ARM (~2000).
   DISARM: SwA to SAFE (~1000) -> the motors stop at once, in every mode (kill switch).

   Automatic safety:
     radio lost in the air ............ LAND (on the ground: motors stop)
     battery critical in the air ...... LAND (you can still steer, not cancel)
     barometer lost in ALT HOLD ....... LAND
     tilt over 70 deg / MPU6050 lost .. motors stop
     throttle down on the ground 15 s . motors stop

   Settings: config.h       Documentation: docs/ in the repository
   Arduino IDE, Tools menu: Board "ESP32S3 Dev Module", Flash Size "16MB (128Mb)",
     Partition Scheme "16M Flash (3MB APP/9.9MB FATFS)", PSRAM "OPI PSRAM".

   Motor layout (seen from above, front at the top):
        M4 (CW)    M2 (CCW)
             \  /
             /  \
        M3 (CCW)   M1 (CW)
*/

#include <Arduino.h>
#include <Wire.h>
#include "esp_log.h"

#include "config.h"
#include "util.h"
#include "rc_ibus.h"
#include "motors.h"
#include "pid.h"
#include "imu_mpu6050.h"
#include "baro_bmp280.h"
#include "altitude.h"
#include "battery.h"
#include "status_beeper.h"
#include "flight_log.h"

#define FC_VERSION "3.0"

// ---------------- Constants ----------------
constexpr uint32_t LOOP_US          = 4000;    // 250 Hz control loop
constexpr uint16_t THR_ARM_MAX      = 1050;    // throttle stick must be below this to arm
constexpr float    CRASH_ANGLE      = 70.0f;   // deg
constexpr uint16_t AIRBORNE_THR     = 1350;    // throttle above this for 0.5 s = taking off
constexpr float    ALT_MIN_FOR_HOLD = 0.5f;    // m: ALT HOLD needs at least this height
constexpr float    GROUND_ALT       = 0.3f;    // m: below this the drone counts as "on the ground"
constexpr float    AUTO_DISARM_S    = 15.0f;   // throttle down on the ground this long -> disarm
constexpr uint32_t LOG_EVERY        = 250 / LOG_HZ;
static_assert(250 % LOG_HZ == 0, "LOG_HZ must divide 250: use 25, 50, 125 or 250");

// ---------------- Flight state ----------------
// Why the drone is landing. A higher number wins (battery locks the mode switch).
enum LandReason : uint8_t { LAND_NONE = 0, LAND_SWITCH, LAND_BARO, LAND_RADIO, LAND_BATTERY };
const char *const LAND_NAMES[] = {"-", "switch", "barometer lost", "radio lost", "battery critical"};

bool armed = false;
bool armSwitchWasDown = false;    // SwA must be seen on SAFE before an arm is accepted
FlightMode flightMode = MODE_MANUAL;
```

```
LandReason landReason = LAND_NONE;
FlightMode lastSwitchMode = MODE_MANUAL;   // last accepted SwC position
FlightMode swCandidate = MODE_MANUAL;      // SwC position waiting to be stable
uint32_t swCandSince = 0;
bool radioWasOk = true;
bool airborne = false;
float airTimer = 0, idleTimer = 0;
float lastThrOut = MOTOR_OFF;              // collective throttle sent to the mixer (us)
float wantRollLog = 0, wantPitchLog = 0;   // wanted angles, for the log
float blindThr = HOVER_THROTTLE_GUESS;     // LAND without barometer: fixed throttle
bool hoverSeeded = false;
BattState lastBattState = BATT_OK;
bool lostBeeper = false;                   // after a crash / radio loss: beep to find the drone
bool statusPrint = true;

// landing detector (LAND, or ALT HOLD with the throttle stick fully down)
float landTime = 0, landedTimer = 0, minThrTimer = 0, lowAltTimer = 0, blindTime = 0;
double landThrSum = 0;
uint32_t landThrN = 0;
bool groundContact = false;

// flight summary
uint32_t armTimeMs = 0, flightTimeMs = 0;
float maxAlt = 0, minCellV = 9.9f;

// ---------------- Flight notes ----------------
// Events during a flight are kept here and printed after landing
// (printing while armed could pause the control loop when USB is unplugged).
constexpr uint8_t MAX_NOTES = 20;
struct Note { uint32_t ms; char text[60]; };
Note notes[MAX_NOTES];
uint8_t noteCount = 0;

void note(const char *fmt, ...) {
  char buf[60];
  va_list ap;
  va_start(ap, fmt);
  vsnprintf(buf, sizeof(buf), fmt, ap);
  va_end(ap);
  if (armed && noteCount < MAX_NOTES) {               // kept for the flight summary
    notes[noteCount].ms = millis() - armTimeMs;
    memcpy(notes[noteCount].text, buf, sizeof(buf));
    noteCount++;
  }
  if (!armed && DEBUG) Serial.println(buf);          // on the ground: print right away
}

void printFlightSummary() {
  if (!DEBUG || armTimeMs == 0) return;
  Serial.printf("\n---- Last flight: %.1f s", flightTimeMs / 1000.0f);
  if (maxAlt > 0.05f) Serial.printf(", highest %.1f m", maxAlt);
  if (minCellV < 9.0f) Serial.printf(", lowest %.2f V/cell under load, now %d%%", minCellV, battPct);
  if (CURRENT_PIN >= 0) Serial.printf(", %.0f mAh used", mAhUsed);
  Serial.println(F(" ----"));
  for (uint8_t i = 0; i < noteCount; i++)
    Serial.printf("  %7.1f s  %s\n", notes[i].ms / 1000.0f, notes[i].text);
  if (noteCount == MAX_NOTES) Serial.println(F("  (later events not kept)"));
}

// ================================================================
//  Arm / disarm
// ================================================================
// Returns why arming is not possible right now, or nullptr when ready.
const char *armProblem() {
  if (!radioOk())                                  return "no radio signal";
  if (!imuOk)                                      return "MPU6050 not found";
  if (rc[CH_THR] > THR_ARM_MAX)                    return "throttle not fully down";
  if (decodeModeSwitch(rc[CH_MODE]) != MODE_MANUAL) return "SwC (CH6) not on MANUAL";
  if (fabsf(accRoll) > 25 || fabsf(accPitch) > 25)  return "drone not level";
  if (battPresent && vCell < CELL_MIN_ARM_V)       return "battery too low, charge it";
  return nullptr;
}
```

```
void disarm(const char *why) {
  if (!armed) return;
  motorsOff();                        // first of all
  note("DISARMED: %s", why);          // still "armed" here -> kept for the summary
  armed = false;
  flightTimeMs = millis() - armTimeMs;
  pidResetAll();
  armSwitchWasDown = false;           // SwA must go back to SAFE before the next arm
  airborne = false;
  flightMode = MODE_MANUAL;
  landReason = LAND_NONE;
  logStop();
  beep(2);
  printFlightSummary();
}

void tryArm() {
  if (!radioOk()) return;
  if (rc[CH_ARM] <= 1700) { armSwitchWasDown = true; return; }
  if (!armSwitchWasDown) return;      // switch was already on ARM: wait until it is flipped
  armSwitchWasDown = false;           // the switch just went to ARM

  const char *problem = armProblem();
  if (problem) {
    note("NO ARM: %s. SwA to SAFE, fix it, try again.", problem);
    beep(3, 60, 60);
    return;
  }
  pidResetAll();
  altZero();                          // height 0 = here
  battResetFlight();
  lastBattState = BATT_OK;
  flightMode = MODE_MANUAL;
  landReason = LAND_NONE;
  lastSwitchMode = swCandidate = MODE_MANUAL;
  airborne = false;
  airTimer = idleTimer = 0;
  radioWasOk = true;
  lostBeeper = false;
  maxAlt = 0;
  minCellV = 9.9f;
  noteCount = 0;
  armTimeMs = millis();
  if (DEBUG) Serial.println(F("ARMED"));
  logStart();
  armed = true;
  beep(1);
}

// ================================================================
//  Modes and automatic landing
// ================================================================
// Throttle that should hold the drone in the air, when leaving MANUAL
float manualHoverBase() {
  return hoverLearnedValid ? hoverThrLearned : lastThrOut;
}

bool onGround() {
  return !airborne || (altValid && altEst < GROUND_ALT);
}

void resetLandDetector() {
  landTime = landedTimer = minThrTimer = lowAltTimer = blindTime = 0;
  landThrSum = 0;
  landThrN = 0;
  groundContact = false;
}

void startLand(LandReason why) {
  if (flightMode == MODE_LAND) {                    // already landing: maybe a stronger reason
    if (why > landReason) { landReason = why; note("landing reason: %s", LAND_NAMES[why]); }
```

```
    return;
  }
  if (onGround()) {                                  // nothing to land: just stop
    static const char *const ON_GROUND[] = {"-", "LAND switch on the ground", "barometer lost on the ground",
                                            "radio lost on the ground", "battery critical on the ground"};
    disarm(ON_GROUND[why]);
    return;
  }
  if (flightMode == MODE_ALT) {
    blindThr = hoverThr + velI - LAND_NOBARO_DROP;   // the speed loop keeps its state
  } else {
    const float base = manualHoverBase();
    altCtrlReset(base);
    blindThr = base - LAND_NOBARO_DROP;
  }
  flightMode = MODE_LAND;
  landReason = why;
  resetLandDetector();
  note("LAND (%s)%s", LAND_NAMES[why], altValid ? "" : " without barometer");
  beep(1);
}

void onModeSwitch(FlightMode want) {
  if (landReason == LAND_BATTERY) {                // battery landing cannot be cancelled
    note("SwC ignored: battery landing");
    beep(3, 60, 60);
    return;
  }
  switch (want) {
    case MODE_MANUAL:
      if (flightMode == MODE_MANUAL) return;
      flightMode = MODE_MANUAL;
      landReason = LAND_NONE;
      note("MANUAL");
      beep(1);
      break;

    case MODE_ALT:
      if (!ALT_HOLD_ENABLED) { note("ALT HOLD refused: disabled in config.h"); beep(3, 60, 60); return; }
      if (!altValid)         { note("ALT HOLD refused: no barometer"); beep(3, 60, 60); return; }
      if (!airborne || altEst < ALT_MIN_FOR_HOLD) {
        note("ALT HOLD refused: climb above %.1f m first", ALT_MIN_FOR_HOLD);
        beep(3, 60, 60);
        return;
      }
      if (flightMode == MODE_LAND) altTarget = altEst;                    // keep the speed loop
      else altCtrlReset(hoverLearnedValid ? hoverThrLearned : lastThrOut);
      flightMode = MODE_ALT;
      landReason = LAND_NONE;
      note("ALT HOLD at %.1f m", altEst);
      beep(1);
      break;

    case MODE_LAND:
      startLand(LAND_SWITCH);
      break;
  }
}

// Hover throttle: the average throttle while flying level and steady.
// Used as the starting point when ALT HOLD or LAND begins.
void learnHover(float dt) {
  if (!airborne || flightMode == MODE_LAND || lastThrOut < THR_ARM_MAX) return;
  if (fabsf(angleRoll) > 15 || fabsf(anglePitch) > 15) return;
  if (altValid && fabsf(velEst) > 0.3f) return;
  const float thrLevel = 1000.0f + (lastThrOut - 1000.0f) * tiltCos();
  if (!hoverSeeded) { hoverThrLearned = thrLevel; hoverSeeded = true; }
  hoverThrLearned += (dt / 3.0f) * (thrLevel - hoverThrLearned);   // ~3 s average
  hoverLearnTime += dt;
  if (hoverLearnTime > 2.0f) hoverLearnedValid = true;
}
```

```
// Landing detector (idea from INAV): the controller pushes the throttle down
// but the drone does not move any more -> it is on the ground.
void landDetect(float dt) {
  landTime += dt;
  if (!altValid) {                                  // no barometer: fixed time
    blindTime += dt;
    if (blindTime >= LAND_NOBARO_TIME_S) disarm("landing finished (no barometer: timer)");
    return;
  }
  if (landTime > 1.0f) { landThrSum += lastThrOut; landThrN++; }
  const float avgThr = landThrN ? (float)(landThrSum / landThrN) : lastThrOut;
  const bool still = fabsf(velEst) < 0.25f && fabsf(rollRateF) < 20 &&
                     fabsf(pitchRateF) < 20 && fabsf(yawRateF) < 20;
  groundContact = landThrN > 0 && still && lastThrOut < avgThr - 20.0f;
  const bool pushedDown = landThrN > 0 && lastThrOut < avgThr - 40.0f;
  // probably touching the ground: lower the throttle faster so it does not skate around.
  // (in the air the drone would start to sink, "still" turns false and this stops)
  if (groundContact) velI = fmaxf(velI - 150.0f * dt, -VEL_I_MAX);

  landedTimer = (landTime > 2.0f && still && pushedDown) ? landedTimer + dt : 0.0f;
  minThrTimer = (lastThrOut <= MOTOR_IDLE + 31 && fabsf(velEst) < 0.5f) ? minThrTimer + dt : 0.0f;
  if (altEst < LAND_SLOW_ALT) lowAltTimer += dt;

  if (landedTimer >= 1.0f)              disarm("landed");
  else if (minThrTimer >= 3.0f)         disarm("landed (throttle at minimum)");
  else if (lowAltTimer >= LAND_TIMEOUT_S) disarm("landing timeout");
}

void updateAirborne(float dt) {
  const bool lifting = lastThrOut >= AIRBORNE_THR ||
                       (altValid && altEst > 0.7f && lastThrOut > MOTOR_IDLE + 100);
  airTimer = lifting ? airTimer + dt : 0.0f;
  if (!airborne && airTimer >= 0.5f) { airborne = true; note("airborne"); }

  // MANUAL with the throttle stick down: on the ground (it cannot fly at idle)
  idleTimer = (flightMode == MODE_MANUAL && rc[CH_THR] < THR_ARM_MAX) ? idleTimer + dt : 0.0f;
  if (airborne && idleTimer >= 1.0f) { airborne = false; note("on the ground"); }
  if (!airborne && idleTimer >= AUTO_DISARM_S) disarm("throttle down for 15 s");
}

// Everything that can change the flight mode or stop the motors. Runs every loop while armed.
void updateFlightState(float dt) {
  const bool radio = radioOk();
  const uint32_t now = millis();

  // 1) kill switch: works in every mode
  if (radio && rc[CH_ARM] <= 1700) { disarm("SwA to SAFE (kill switch)"); return; }

  // 2) crash or sensor failure
  if (fabsf(angleRoll) > CRASH_ANGLE || fabsf(anglePitch) > CRASH_ANGLE) {
    lostBeeper = airborne;
    disarm("crash: tilted more than 70 deg");
    return;
  }
  if (imuErrors > 20) { lostBeeper = airborne; disarm("MPU6050 stopped answering"); return; }

  // 3) in the air or on the ground?
  updateAirborne(dt);
  if (!armed) return;

  // 4) radio lost / back
  if (!radio && radioWasOk) {
    radioWasOk = false;
    if (airborne) lostBeeper = true;
    startLand(LAND_RADIO);                          // on the ground this disarms
    if (!armed) return;
  }
  if (radio && !radioWasOk) {
    radioWasOk = true;
    lastSwitchMode = swCandidate = decodeModeSwitch(rc[CH_MODE]);
    note("radio back: move SwC to take over");
  }
```

```
  // 5) battery
  if (battState != lastBattState) {
    lastBattState = battState;
    note("battery %s: %.2f V/cell (%d%%)", battState == BATT_CRITICAL ? "CRITICAL" : "LOW", vCell, battPct);
  }
  if (battPresent && battState == BATT_CRITICAL && airborne && landReason != LAND_BATTERY) {
    startLand(LAND_BATTERY);
    if (!armed) return;
  }

  // 6) barometer lost while holding height
  if (flightMode == MODE_ALT && !altValid) startLand(LAND_BARO);

  // 7) mode switch. MANUAL and LAND act at once; the middle (ALT HOLD) must be
  //    stable for 0.15 s (a quick flip from one end to the other passes the middle).
  if (radio) {
    const FlightMode sw = decodeModeSwitch(rc[CH_MODE]);
    if (sw != swCandidate) { swCandidate = sw; swCandSince = now; }
    const bool stable = (sw != MODE_ALT) || (now - swCandSince >= 150);
    if (sw != lastSwitchMode && stable) {
      lastSwitchMode = sw;
      onModeSwitch(sw);
      if (!armed) return;
    }
  }

  // 8) landing detector: in LAND, or in ALT HOLD with the throttle stick fully down
  const bool descending = flightMode == MODE_LAND ||
                          (flightMode == MODE_ALT && radio && rc[CH_THR] < 1100);
  if (descending) landDetect(dt);
  else resetLandDetector();
  if (!armed) return;

  // 9) learn the hover throttle
  learnHover(dt);
}

// ================================================================
//  Flight control: throttle + sticks -> PIDs -> motor mix
// ================================================================
void flightControl(float dt) {
  if (!armed) { motorsOff(); lastThrOut = MOTOR_OFF; return; }
  const bool radio = radioOk();

  // 1) collective throttle
  float thr;
  if (flightMode == MODE_MANUAL) {
    const uint16_t stickThr = constrain(rc[CH_THR], 1000, THROTTLE_MAX);
    if (stickThr < THR_ARM_MAX) {            // throttle down: idle, no corrections
      pidResetAll();
      motorsAll(MOTOR_IDLE);
      lastThrOut = MOTOR_IDLE;
      wantRollLog = wantPitchLog = 0;
      return;
    }
    thr = stickThr;
  } else if (altValid) {                     // ALT HOLD and LAND: the height controller decides
    thr = altCtrlUpdate(flightMode == MODE_ALT ? altHoldVelTarget() : landVelTarget(), dt);
    if (flightMode == MODE_LAND) blindThr = hoverThr + velI - LAND_NOBARO_DROP;   // in case the baro fails
  } else {                                   // LAND without barometer: a bit under hover
    thr = clampf(blindThr, MOTOR_IDLE + 30.0f, THROTTLE_MAX);
    thr = 1000.0f + (thr - 1000.0f) / fmaxf(tiltCos(), 0.7f);
  }
  lastThrOut = thr;

  // 2) sticks -> wanted angles and yaw rate (centred when the radio is lost)
  float sRoll = 0, sPitch = 0, sYaw = 0;
  if (radio) { sRoll = stick(rc[CH_ROLL]); sPitch = stick(rc[CH_PITCH]); sYaw = stick(rc[CH_YAW]); }
  const float wantRoll  =  sRoll  * MAX_ANGLE_DEG;   // right stick right   -> right side down
  const float wantPitch = -sPitch * MAX_ANGLE_DEG;   // right stick forward -> nose down
  const float wantYaw   =  sYaw   * MAX_YAW_RATE;    // left stick right    -> nose right
```

```
  wantRollLog = wantRoll;
  wantPitchLog = wantPitch;

  // 3) angle error -> wanted rotation speed (self-levelling)
  const float rollRateSet  = clampf(LEVEL_GAIN * (wantRoll  - angleRoll),  -250.0f, 250.0f);
  const float pitchRateSet = clampf(LEVEL_GAIN * (wantPitch - anglePitch), -250.0f, 250.0f);

  // 4) no I-term wind-up on the ground (it can flip the drone at take-off or touchdown)
  if (!airborne) {
    pidRoll.i = pidPitch.i = pidYaw.i = 0;
  } else if (groundContact) {
    pidRoll.i *= 0.95f; pidPitch.i *= 0.95f; pidYaw.i *= 0.95f;
  }

  // 5) rate PIDs and mixer
  const float outRoll  = pidRun(pidRoll,  rollRateSet  - rollRateF,  P_ROLL, I_ROLL, D_ROLL);
  const float outPitch = pidRun(pidPitch, pitchRateSet - pitchRateF, P_ROLL, I_ROLL, D_ROLL);
  const float outYaw   = pidRun(pidYaw,   wantYaw      - yawRateF,   P_YAW,  I_YAW,  D_YAW);
  motorsMix(thr, outRoll, outPitch, outYaw);
}

// ================================================================
//  Flight log record (50 times per second while armed)
// ================================================================
int16_t sat16(float v) { return (int16_t)lrintf(clampf(v, -32767.0f, 32767.0f)); }

void logRecord() {
  LogRec r;
  r.t_ms = millis();
  r.roll_cd = sat16(angleRoll * 100);
  r.pitch_cd = sat16(anglePitch * 100);
  r.rollSp_cd = sat16(wantRollLog * 100);
  r.pitchSp_cd = sat16(wantPitchLog * 100);
  r.yawRate_dd = sat16(yawRateF * 10);
  r.alt_cm = sat16(altEst * 100);
  r.altTgt_cm = sat16(flightMode == MODE_ALT ? altTarget * 100 : altEst * 100);
  r.baroAlt_cm = sat16(baroAlt * 100);
  r.vz_cms = sat16(velEst * 100);
  r.thrIn = rc[CH_THR];
  r.thrOut = (uint16_t)lastThrOut;
  for (uint8_t m = 0; m < 4; m++) r.m[m] = motorOut[m];
  r.vbat_mv = (uint16_t)(vbat * 1000);
  r.vrest_mv = (uint16_t)(vRest * 1000);
  r.amps_ca = sat16(battAmps * 100);
  r.battPct = battPct < 0 ? 0 : (uint8_t)battPct;
  r.spare = 0;
  r.mode = flightMode;
  uint8_t f = LOGF_ARMED;
  if (airborne) f |= LOGF_AIRBORNE;
  if (radioOk()) f |= LOGF_RADIO_OK;
  if (battState == BATT_LOW) f |= LOGF_BATT_LOW;
  if (battState == BATT_CRITICAL) f |= LOGF_BATT_CRIT;
  if (baroOk) f |= LOGF_BARO_OK;
  if (landReason == LAND_RADIO) f |= LOGF_LAND_RADIO;
  if (landReason == LAND_BATTERY) f |= LOGF_LAND_BATT;
  r.flags = f;
  logAdd(r);
}

// ================================================================
//  Buzzer + LED
// ================================================================
void statusUpdate(uint32_t now) {
  // repeating beeps, the most important first
  BeepRepeat rep = REP_NONE;
  if (armed && flightMode == MODE_LAND)                rep = REP_LANDING;
  else if (battPresent && battState == BATT_CRITICAL)  rep = REP_BATT_CRIT;
  else if (battPresent && battState == BATT_LOW)       rep = REP_BATT_LOW;
  else if (!armed && lostBeeper)                       rep = REP_LOST;
  beepRepeat = rep;
  beepUpdate(now);
```

```
  const bool blink = (now / 250) % 2;
  const bool battLow = battPresent && battState == BATT_LOW;
  if (!imuOk || (battPresent && battState == BATT_CRITICAL)) {
    ledWrite(blink ? 60 : 0, 0, 0);                          // red blink: problem
  } else if (armed) {
    if (flightMode == MODE_LAND)       ledWrite(blink ? 60 : 0, blink ? 20 : 0, 0);  // orange blink
    else if (battLow && blink)         ledWrite(50, 40, 0);   // yellow flashes: battery low
    else if (flightMode == MODE_ALT)   ledWrite(35, 0, 50);   // purple: ALT HOLD
    else                               ledWrite(0, 0, 60);    // blue: MANUAL
  } else {
    if (battLow && blink)              ledWrite(50, 40, 0);
    else if (armProblem())             ledWrite(0, blink ? 40 : 0, 0);   // green blink: not ready
    else                               ledWrite(0, 40, 0);    // green: ready to arm
  }
}

// ================================================================
//  Serial Monitor (115200 baud) - only while DISARMED
// ================================================================
void printHelp() {
  Serial.println(F("\nCommands (only while DISARMED; type, then Enter):"));
  Serial.println(F("  h    this help"));
  Serial.println(F("  i    info: sensors, battery, hover throttle, log storage, last flight"));
  Serial.println(F("  s    live status lines on/off"));
  Serial.println(F("  l    list saved flight logs"));
  Serial.println(F("  d    print the newest log as CSV   (d3 = log 3)"));
  Serial.println(F("  X    erase ALL logs"));
  Serial.println(F("  z    set height 0 here"));
  Serial.println(F("tools/flight_log.py downloads and plots the logs for you.\n"));
}

void printInfo() {
  Serial.printf("\n---- Flight controller v%s ----\n", FC_VERSION);
  Serial.printf("MPU6050 ........ %s\n", imuOk ? "OK" : "NOT FOUND (run sketch 3)");
  if (baroOk)
    Serial.printf("BMP280 ......... OK at 0x%02X, %.2f hPa, %.1f C, height %+.2f m\n",
                  bmpAddr, baroPressurePa / 100.0f, baroTempC, altEst);
  else
    Serial.printf("BMP280 ......... %s -> no ALT HOLD, landing without height control\n",
                  ALT_HOLD_ENABLED ? "NOT FOUND" : "off in config.h");
  if (battPresent)
    Serial.printf("Battery ........ %.2f V = %.2f V/cell, %d%% (%s), sag %.2f V per unit load\n", vbat,
vCell,
                  battPct, battState == BATT_CRITICAL ? "CRITICAL" : battState == BATT_LOW ? "LOW" : "OK",
sagK);
  else
    Serial.printf("Battery ........ no reading (%.2f V): LiPo unplugged, or check H5 / VBAT_CAL\n", vbat);
  Serial.printf("Hover throttle . %.0f us (%s)\n", hoverThrLearned,
                hoverLearnedValid ? "learned in flight" : "guess from config.h");
  Serial.printf("ESC signal ..... %d Hz, off %d us, idle %d us\n", ESC_FREQ_HZ, MOTOR_OFF, MOTOR_IDLE);
  if (logCap)
    Serial.printf("Log memory ..... %s, room for %.1f min at %d Hz\n",
                  logCap > 5000 ? "PSRAM" : "small RAM buffer (enable Tools > PSRAM > OPI PSRAM)",
                  logCap / (float)LOG_HZ / 60.0f, LOG_HZ);
  else
    Serial.println(F("Log memory ..... none (logging off)"));
  if (fsOk) {
    int newest, oldest, count;
    logScan(newest, oldest, count);
    Serial.printf("Log flash ...... %d saved, %lu KB free\n", count, (unsigned long)(FFat.freeBytes() /
1024));
  } else {
    Serial.println(F("Log flash ...... NOT AVAILABLE: Tools > Partition Scheme > 16M Flash (3MB APP/9.9MB
FATFS)"));
  }
  printFlightSummary();
  Serial.println();
}

void printStatus() {
  const char *problem = armProblem();
  static const char *const SWC[3] = {"MANUAL", "ALT", "LAND"};
```

```
  char batt[24];
  if (battPresent) snprintf(batt, sizeof(batt), "%.2f V %d%%", vbat, battPct);
  else snprintf(batt, sizeof(batt), "no LiPo");
  Serial.printf("%s | roll %+5.1f pitch %+5.1f | THR %4u SwA %-4s SwC %-6s | h %+5.2f m | %s%s%s\n",
                problem ? "WAIT " : "READY", angleRoll, anglePitch, rc[CH_THR],
                rc[CH_ARM] > 1700 ? "ARM" : "SAFE", SWC[decodeModeSwitch(rc[CH_MODE])],
                altValid ? altEst : 0.0f, batt, problem ? "  <- " : "", problem ? problem : "");
}

void runCommand(const char *cmd) {
  switch (cmd[0]) {
    case 'h': case '?': printHelp(); break;
    case 'i': printInfo(); break;
    case 's':
      statusPrint = !statusPrint;
      Serial.printf("Live status %s\n", statusPrint ? "ON" : "OFF");
      break;
    case 'l': logList(); break;
    case 'd': logDump(cmd[1] ? atoi(cmd + 1) : -1); break;
    case 'X': logEraseAll(); break;
    case 'z': altZero(); Serial.println(F("Height set to 0 here.")); break;
    default: Serial.println(F("Unknown command. Type h for help."));
  }
}

void handleSerial() {
  static char line[16];
  static uint8_t len = 0;
  static uint32_t lastCharMs = 0;
  while (Serial.available()) {
    const char c = Serial.read();
    lastCharMs = millis();
    if (c == '\n' || c == '\r') {
      if (len) { line[len] = 0; runCommand(line); len = 0; }
    } else if (len < sizeof(line) - 1) {
      line[len++] = c;
    }
  }
  if (len && millis() - lastCharMs > 150) { line[len] = 0; runCommand(line); len = 0; }   // "No line ending"
}

void saveLogNow() {
  ledWrite(40, 40, 40);                                 // white: saving, do not unplug
  if (DEBUG) Serial.println(F("Saving the flight log to flash... wait for the long beep."));
  if (!logSave() && DEBUG) Serial.println(F("Log NOT saved (type i)."));
  beep(1, 500, 100);                                    // long beep = ready again
}

// ================================================================
//  Setup / loop
// ================================================================
void setup() {
  escInit();                            // FIRST: the ESCs must see "off" right away
  statusBegin();
  ledWrite(40, 40, 0);                  // yellow: starting
  Serial.begin(115200);
  rcBegin();
  analogReadResolution(12);
  esp_log_level_set("i2c.master", ESP_LOG_NONE);   // no driver error spam on the serial port
  esp_log_level_set("i2c", ESP_LOG_NONE);
  Wire.begin(I2C_SDA_PIN, I2C_SCL_PIN, 400000);
  Wire.setTimeOut(3);
  delay(1000);

  if (DEBUG) Serial.printf("\n========== SKETCH 4: FLIGHT CONTROLLER v%s ==========\n", FC_VERSION);
  imuOk = mpuInit();
  if (imuOk) {
    calibrateGyro();
    readImu();
    angleRoll = accRoll;
    anglePitch = accPitch;
  } else if (DEBUG) {
```

```
    Serial.println(F("MPU6050 NOT FOUND -> arming is blocked (run sketch 3)."));
  }

  baroOk = ALT_HOLD_ENABLED && baroInit();
  altZero();
  if (DEBUG) {
    if (baroOk) Serial.printf("BMP280 OK at 0x%02X (%.1f hPa)\n", bmpAddr, baroPressurePa / 100.0f);
    else Serial.println(F("BMP280 not used: no ALT HOLD, landing without height control."));
  }

  if (DEBUG) Serial.println(F("Log storage... (the very first start can take 30 s)"));
  logBegin();
  for (uint8_t i = 0; i < 10; i++) battUpdate(0.1f, false);

  if (DEBUG) {
    printInfo();
    Serial.println(F("Ready. To arm: SwC on MANUAL, throttle DOWN, then SwA to ARM.  Type h for help."));
  }
  beep(1, 500, 100);                  // long beep = ready
}

void loop() {
  static uint32_t loopTimer = micros();
  static uint32_t loopCount = 0;
  static float battDt = 0;

  // run at a steady 250 Hz
  while (micros() - loopTimer < LOOP_US) { }
  float dt = (micros() - loopTimer) / 1e6f;
  loopTimer = micros();
  if (dt > 0.02f) dt = 0.02f;
  loopCount++;
  const uint32_t now = millis();

  // sensors
  rcUpdate();
  accTrust = armed ? 0.002f : 0.02f;                              // see imu_mpu6050.h
  if (imuOk) updateAttitude(dt);
  const bool newBaro = (loopCount % 5 == 0) && baroUpdate();      // 50 Hz
  altUpdate(dt, newBaro);
  battDt += dt;
  if (loopCount % 25 == 0) { battUpdate(battDt, armed); battDt = 0; }   // 10 Hz

  // decisions and motors
  if (armed) updateFlightState(dt);
  else tryArm();
  flightControl(dt);

  if (armed) {
    if (altValid && altEst > maxAlt) maxAlt = altEst;
    if (battPresent && vCellRaw < minCellV) minCellV = vCellRaw;
    if (loopCount % LOG_EVERY == 0) logRecord();
  }
  statusUpdate(now);

  // only on the ground: save the log, Serial Monitor
  if (!armed) {
    if (logPendingSave && shot.remaining == 0) saveLogNow();
    handleSerial();
    if (DEBUG && statusPrint && loopCount % 125 == 0) printStatus();   // every 0.5 s
  }
}
```

D2. 4_flight_controller / config.h

```
#pragma once
// ================================================================
//  USER SETTINGS  —  the only file you normally need to edit.
```

```
//  Everything is explained in docs/ (see docs/README.md for the index).
// ======================================================================

// ----------------------------------------------------------------------
//  1. From sketch 3 (guide Step 8): how the MPU6050 sits on the frame
// ----------------------------------------------------------------------
#define IMU_ROTATION      0          // X arrow on the GY-521: 0 forward, 90 right, 180 back, 270 left
#define IMU_UPSIDE_DOWN   false      // true only if the chip faces the ground
#define ACC_ROLL_TRIM     0.00f      // the two numbers sketch 3 printed after 'l'
#define ACC_PITCH_TRIM    0.00f

// ----------------------------------------------------------------------
//  2. From sketch 1 (guide Steps 2-5): ESC signal
// ----------------------------------------------------------------------
#define ESC_FREQ_HZ       250        // fastest speed that armed + ran smoothly ('f' in sketch 1)
#define MOTOR_OFF         1000       // "off" pulse that lets the ESCs arm ('o' in sketch 1)
#define MOTOR_IDLE        1100       // armed with throttle down: motors spin slowly
#define MOTOR_MAX         2000
#define THROTTLE_MAX      1800       // keeps room for corrections at full throttle

// ----------------------------------------------------------------------
//  3. Pins (your PCB)
// ----------------------------------------------------------------------
static const uint8_t MOTOR_PINS[4] = {10, 11, 12, 13};   // H6 pins 1..4 = M1..M4
#define IBUS_RX_PIN       18         // H2 pin 2
#define I2C_SDA_PIN       9          // MPU6050 + BMP280
#define I2C_SCL_PIN       8
#define VBAT_PIN          1          // H5: battery divider R3 10k / R4 3.3k
#define BUZZER_PIN        17         // H3 pin 3 (the unused GPS connector). -1 = no buzzer
#define BUZZER_ACTIVE_HIGH true      // "high level trigger" buzzer module
#define BUZZER_PASSIVE    false      // true = bare piezo that needs a tone
#define RGB_LED_PIN       48         // RGB LED on the DevKit (38 on some boards). -1 = off
#define CURRENT_PIN       -1         // optional current sensor output on an ADC1 pin (e.g. 2). -1 = none

// ----------------------------------------------------------------------
//  4. Radio channels (FS-i6X, i-BUS). Channel numbers start at 0 here.
// ----------------------------------------------------------------------
#define CH_ROLL           0          // CH1
#define CH_PITCH          1          // CH2
#define CH_THR            2          // CH3
#define CH_YAW            3          // CH4
#define CH_ARM            4          // CH5 = SwA : ~2000 ARM, ~1000 SAFE (= KILL switch)
#define CH_MODE           5          // CH6 = SwC : ~1000 MANUAL, ~1500 ALT HOLD, ~2000 LAND

// ----------------------------------------------------------------------
//  5. How the drone answers the sticks (MANUAL mode = self-levelling)
// ----------------------------------------------------------------------
constexpr float MAX_ANGLE_DEG  = 30.0f;   // full right stick = this tilt
constexpr float LEVEL_GAIN     = 5.0f;    // how hard it pulls back to level
constexpr float MAX_YAW_RATE   = 150.0f;  // deg/s at full yaw stick

// ----------------------------------------------------------------------
//  6. Attitude PID (start values for F330 + A2212 2200KV + 6" props)
// ----------------------------------------------------------------------
constexpr float P_ROLL  = 1.3f,  I_ROLL = 0.04f, D_ROLL = 18.0f;   // pitch uses the same
constexpr float P_YAW   = 4.0f,  I_YAW  = 0.02f, D_YAW  = 0.0f;
constexpr float PID_MAX = 400.0f;

// ----------------------------------------------------------------------
//  7. Battery management
// ----------------------------------------------------------------------
#define BATTERY_CELLS     3
constexpr float VBAT_DIVIDER     = (10000.0f + 3300.0f) / 3300.0f;   // (R3 + R4) / R4
constexpr float VBAT_CAL         = 1.00f;   // = multimeter volts / volts shown by the drone
// The thresholds use the RESTING voltage per cell (sag compensated, see battery.h):
constexpr float CELL_WARN_V      = 3.75f;   // ~25 % left, 5 s -> beep every second, yellow LED
constexpr float CELL_CRITICAL_V  = 3.65f;   // ~7 % left, 5 s  -> AUTOMATIC LANDING
constexpr float CELL_ABS_MIN_V   = 3.30f;   // measured under load, 5 s -> also AUTOMATIC LANDING
constexpr float CELL_MIN_ARM_V   = 3.75f;   // at rest: refuse to arm below this
constexpr float CURRENT_A_PER_V  = 0.0f;    // current sensor: amps per volt of output
constexpr float CURRENT_OFFSET_V = 0.0f;    // current sensor: output volts at 0 A
```

```
// ---------------------------------------------------------------------
//  8. Altitude hold (BMP280 barometer + accelerometer)
// ---------------------------------------------------------------------
#define ALT_HOLD_ENABLED  true
constexpr float ALT_P            = 0.8f;    // (m/s) of climb per metre of altitude error
constexpr float VEL_P            = 80.0f;   // throttle us per (m/s) of vertical speed error
constexpr float VEL_I            = 40.0f;   // throttle us per (m/s x s): fixes hover throttle error
constexpr float VEL_I_MAX        = 150.0f;  // limit of that correction (us)
constexpr float ALT_MAX_CLIMB    = 0.8f;    // m/s with the throttle stick fully up (ALT HOLD)
constexpr float ALT_MAX_DESCENT  = 0.6f;    // m/s with the throttle stick fully down
constexpr int   ALT_STICK_DEADBAND = 100;   // us around 1500 where the drone just holds height
constexpr float HOVER_THROTTLE_GUESS = 1500.0f;  // first guess; learned during flight

// ---------------------------------------------------------------------
//  9. Automatic landing
// ---------------------------------------------------------------------
constexpr float LAND_SPEED_FAST  = 0.6f;    // m/s while higher than LAND_SLOW_ALT
constexpr float LAND_SPEED_SLOW  = 0.3f;    // m/s for the last metres
constexpr float LAND_SLOW_ALT    = 1.5f;    // m above the take-off point
constexpr float LAND_TIMEOUT_S   = 30.0f;   // safety net: s below LAND_SLOW_ALT, then motors stop
constexpr float LAND_NOBARO_DROP = 40.0f;   // no barometer: throttle this far below hover...
constexpr float LAND_NOBARO_TIME_S = 20.0f; // ...for this long, then motors stop (firm landing!)

// ---------------------------------------------------------------------
// 10. Flight log (stored in PSRAM during flight, saved to flash after)
// ---------------------------------------------------------------------
#define LOG_ENABLED       true
#define LOG_HZ            50
#define LOG_MAX_FILES     20

// Print status / events on the Serial Monitor (never slows the loop in flight)
#define DEBUG             true
```

D3. 4_flight_controller / util.h

```
#pragma once
// Small helpers shared by every module.
#include <Wire.h>

constexpr float RAD2DEG = 57.29578f;
constexpr float DEG2RAD = 0.0174533f;
constexpr float GRAVITY = 9.80665f;

inline float clampf(float v, float lo, float hi) { return v < lo ? lo : (v > hi ? hi : v); }

// ---- I2C (MPU6050 and BMP280 share the bus) ----
bool i2cReadBytes(uint8_t addr, uint8_t reg, uint8_t *buf, uint8_t n) {
  Wire.beginTransmission(addr);
  Wire.write(reg);
  if (Wire.endTransmission(false) != 0) return false;
  if (Wire.requestFrom(addr, n) != n) return false;
  for (uint8_t i = 0; i < n; i++) buf[i] = Wire.read();
  return true;
}

int i2cReadReg(uint8_t addr, uint8_t reg) {
  uint8_t v;
  return i2cReadBytes(addr, reg, &v, 1) ? v : -1;
}

bool i2cWriteReg(uint8_t addr, uint8_t reg, uint8_t val) {
  Wire.beginTransmission(addr);
  Wire.write(reg);
  Wire.write(val);
  return Wire.endTransmission() == 0;
}
```

D4. 4_flight_controller / rc_ibus.h

```
#pragma once
// FlySky i-BUS receiver (FS-iA10B "i-BUS SERVO" port) on UART1, receive only.
// Frame: 0x20 0x40, 14 channels x 2 bytes (little-endian), 2-byte checksum.

constexpr uint32_t FAILSAFE_MS = 250;     // no valid frame for this long = radio lost

uint16_t rc[14] = {1500, 1500, 1000, 1500, 1000, 1000, 1500, 1500,
                   1500, 1500, 1500, 1500, 1500, 1500};
uint32_t lastFrameMs = 0;

void rcBegin() {
  Serial1.begin(115200, SERIAL_8N1, IBUS_RX_PIN, -1);
}

void rcUpdate() {
  static uint8_t buf[32];
  static uint8_t idx = 0;
  while (Serial1.available()) {
    uint8_t b = Serial1.read();
    if (idx == 0) { if (b == 0x20) buf[idx++] = b; continue; }
    if (idx == 1) {
      if (b == 0x40) buf[idx++] = b;
      else if (b != 0x20) idx = 0;
      continue;
    }
    buf[idx++] = b;
    if (idx == 32) {
      idx = 0;
      uint16_t sum = 0xFFFF;
      for (uint8_t i = 0; i < 30; i++) sum -= buf[i];
      if (sum == (uint16_t)(buf[30] | (buf[31] << 8))) {
        for (uint8_t ch = 0; ch < 14; ch++) {
          uint16_t v = (buf[2 + ch * 2] | (buf[3 + ch * 2] << 8)) & 0x0FFF;
          rc[ch] = v < 1000 ? 1000 : (v > 2000 ? 2000 : v);
        }
        lastFrameMs = millis();
      }
    }
  }
}

bool radioOk() {
  return lastFrameMs != 0 && (millis() - lastFrameMs) < FAILSAFE_MS;
}

// Stick position -1.0 .. +1.0 with a small dead zone around the centre
float stick(uint16_t v) {
  int d = (int)v - 1500;
  if (abs(d) <= 8) return 0.0f;
  return clampf((d > 0 ? d - 8 : d + 8) / 492.0f, -1.0f, 1.0f);
}

// 3-position mode switch (SwC on CH6): ~1000 MANUAL, ~1500 ALT HOLD, ~2000 LAND
enum FlightMode : uint8_t { MODE_MANUAL = 0, MODE_ALT = 1, MODE_LAND = 2 };
const char *const MODE_NAMES[3] = {"MANUAL", "ALT HOLD", "LAND"};

FlightMode decodeModeSwitch(uint16_t v) {
  if (v < 1300) return MODE_MANUAL;
  if (v > 1700) return MODE_LAND;
  return MODE_ALT;
}
```

D5. 4_flight_controller / motors.h

```
#pragma once
// ESC output (LEDC hardware PWM) and the Quad-X motor mixer.
//
//   Motor layout seen from above, front at the top:
//        M4 (CW)    M2 (CCW)
//             \  /
//             /  \    (X frame)
//        M3 (CCW)   M1 (CW)
#include "driver/gpio.h"

constexpr uint8_t ESC_RES_BITS = 14;
uint16_t motorOut[4] = {MOTOR_OFF, MOTOR_OFF, MOTOR_OFF, MOTOR_OFF};

void escWriteUs(uint8_t m, uint16_t us) {
  const uint32_t periodUs = 1000000UL / ESC_FREQ_HZ;
  const uint32_t maxDuty  = (1UL << ESC_RES_BITS) - 1;
  const uint32_t duty = (uint32_t)us * maxDuty / periodUs;
#if ESP_ARDUINO_VERSION_MAJOR >= 3
  ledcWrite(MOTOR_PINS[m], duty);
#else
  ledcWrite(m, duty);
#endif
}

void motorsAll(uint16_t us) {
  for (uint8_t m = 0; m < 4; m++) { motorOut[m] = us; escWriteUs(m, us); }
}

void motorsOff() { motorsAll(MOTOR_OFF); }

// Must run FIRST in setup(): the ESCs see "off" right away, and the motors
// take LEDC channels 0-3 before anything else (the buzzer uses channel 4).
void escInit() {
  for (uint8_t m = 0; m < 4; m++) {
#if ESP_ARDUINO_VERSION_MAJOR >= 3
    ledcAttach(MOTOR_PINS[m], ESC_FREQ_HZ, ESC_RES_BITS);
#else
    ledcSetup(m, ESC_FREQ_HZ, ESC_RES_BITS);
    ledcAttachPin(MOTOR_PINS[m], m);
#endif
    gpio_set_drive_capability((gpio_num_t)MOTOR_PINS[m], GPIO_DRIVE_CAP_3);
  }
  motorsOff();
}

// thr = collective throttle (us); roll/pitch/yaw = corrections from the PIDs
//   +roll  = more power on the LEFT motors  (M3, M4)
//   +pitch = more power on the FRONT motors (M2, M4)
//   +yaw   = more power on the CCW motors   (M2, M3)
void motorsMix(float thr, float roll, float pitch, float yaw) {
  const float mix[4] = {
    thr - roll - pitch - yaw,   // M1 rear-right  CW
    thr - roll + pitch + yaw,   // M2 front-right CCW
    thr + roll - pitch + yaw,   // M3 rear-left   CCW
    thr + roll + pitch - yaw    // M4 front-left  CW
  };
  for (uint8_t m = 0; m < 4; m++) {
    motorOut[m] = (uint16_t)clampf(mix[m], (float)MOTOR_IDLE, (float)MOTOR_MAX);
    escWriteUs(m, motorOut[m]);
  }
}
```

D6. 4_flight_controller / pid.h

```
#pragma once
// Rate PID used for roll, pitch and yaw.
// Runs at a fixed 250 Hz, so I and D are "per loop" (no dt), like YMFC-AL.

struct Pid { float i; float lastErr; };
Pid pidRoll = {0, 0}, pidPitch = {0, 0}, pidYaw = {0, 0};

float pidRun(Pid &p, float err, float kp, float ki, float kd) {
  p.i = clampf(p.i + ki * err, -PID_MAX, PID_MAX);
  const float out = kp * err + p.i + kd * (err - p.lastErr);
  p.lastErr = err;
  return clampf(out, -PID_MAX, PID_MAX);
}

void pidResetAll() {
  pidRoll = {0, 0};
  pidPitch = {0, 0};
  pidYaw = {0, 0};
}
```

D7. 4_flight_controller / imu_mpu6050.h

```
#pragma once
// MPU6050 (GY-521) + attitude estimation (complementary filter).
// Conventions after mapAxes():
//   roll  + = right side down      pitch + = nose up      yaw + = nose turning right

constexpr uint8_t MPU_ADDR = 0x68;
constexpr float GYRO_LSB = 65.5f;     // +-500 deg/s
constexpr float ACC_LSB  = 4096.0f;   // +-8 g

float gyroOff[3] = {0, 0, 0};
float rollRate = 0, pitchRate = 0, yawRate = 0;      // deg/s, raw
float rollRateF = 0, pitchRateF = 0, yawRateF = 0;   // deg/s, smoothed for the PID
float accF = 0, accL = 0, accU = 1;                  // g, drone frame: forward / left / up
float accRoll = 0, accPitch = 0;                     // deg, from the accelerometer only
float angleRoll = 0, anglePitch = 0;                 // deg, fused estimate
uint16_t imuErrors = 0;
bool imuOk = false;
// How much the accelerometer corrects the angles each loop. The main sketch uses a
// fast 0.02 while disarmed (no vibration: settle quickly) and 0.002 while flying.
float accTrust = 0.02f;

bool mpuInit() {
  if (i2cReadReg(MPU_ADDR, 0x75) < 0) return false;
  i2cWriteReg(MPU_ADDR, 0x6B, 0x01);   // wake up, clock = gyro PLL
  delay(50);
  i2cWriteReg(MPU_ADDR, 0x1A, 0x03);   // digital low-pass ~44 Hz (cuts motor vibration)
  i2cWriteReg(MPU_ADDR, 0x1B, 0x08);   // gyro  +-500 deg/s
  i2cWriteReg(MPU_ADDR, 0x1C, 0x10);   // accel +-8 g
  return true;
}

bool readRaw(int16_t r[6]) {          // ax ay az gx gy gz
  uint8_t b[14];
  if (!i2cReadBytes(MPU_ADDR, 0x3B, b, 14)) return false;
  r[0] = (int16_t)(b[0] << 8 | b[1]);
  r[1] = (int16_t)(b[2] << 8 | b[3]);
  r[2] = (int16_t)(b[4] << 8 | b[5]);
  r[3] = (int16_t)(b[8] << 8 | b[9]);
  r[4] = (int16_t)(b[10] << 8 | b[11]);
  r[5] = (int16_t)(b[12] << 8 | b[13]);
  return true;
}
```

```
// chip axes -> drone axes (forward / left / up). Same as sketch 3.
void mapAxes(float x, float y, float z, float &fwd, float &left, float &up) {
  switch (IMU_ROTATION) {
    case 90:  fwd =  y; left = -x; break;
    case 180: fwd = -x; left = -y; break;
    case 270: fwd = -y; left =  x; break;
    default:  fwd =  x; left =  y; break;
  }
  up = z;
  if (IMU_UPSIDE_DOWN) { left = -left; up = -up; }
}

bool readImu() {
  int16_t r[6];
  if (!readRaw(r)) return false;
  float gf, gl, gu;
  mapAxes((r[3] - gyroOff[0]) / GYRO_LSB, (r[4] - gyroOff[1]) / GYRO_LSB,
          (r[5] - gyroOff[2]) / GYRO_LSB, gf, gl, gu);
  rollRate  =  gf;
  pitchRate = -gl;
  yawRate   = -gu;
  mapAxes(r[0] / ACC_LSB, r[1] / ACC_LSB, r[2] / ACC_LSB, accF, accL, accU);
  accRoll  = atan2f(accL, accU) * RAD2DEG - ACC_ROLL_TRIM;
  accPitch = atan2f(accF, sqrtf(accL * accL + accU * accU)) * RAD2DEG - ACC_PITCH_TRIM;
  return true;
}

void calibrateGyro() {
  if (DEBUG) Serial.println(F("Gyro calibration: don't move the drone (8 s)..."));
  double sum[3] = {0, 0, 0};
  int n = 0;
  int16_t r[6];
  for (int i = 0; i < 2000; i++) {
    if (readRaw(r)) { sum[0] += r[3]; sum[1] += r[4]; sum[2] += r[5]; n++; }
    rcUpdate();
    delay(4);
  }
  if (n > 0) for (int k = 0; k < 3; k++) gyroOff[k] = sum[k] / n;
}

// Gyro + accelerometer fusion (complementary filter), called at 250 Hz.
// In flight accTrust = 0.002: the angle follows the gyro and is pulled towards the
// accelerometer with a time constant of 1 / (0.002 * 250 Hz) = 2 s.
void updateAttitude(float dt) {
  if (!readImu()) { if (imuErrors < 1000) imuErrors++; return; }
  imuErrors = 0;

  rollRateF  = 0.7f * rollRateF  + 0.3f * rollRate;
  pitchRateF = 0.7f * pitchRateF + 0.3f * pitchRate;
  yawRateF   = 0.7f * yawRateF   + 0.3f * yawRate;

  angleRoll  += rollRate * dt;
  anglePitch += pitchRate * dt;
  const float s = sinf(yawRate * dt * DEG2RAD);     // yawing swaps roll and pitch a little
  anglePitch -= angleRoll * s;
  angleRoll  += anglePitch * s;
  angleRoll  += accTrust * (accRoll  - angleRoll);
  anglePitch += accTrust * (accPitch - anglePitch);
}

// Vertical acceleration in m/s^2 (up = +), gravity removed. Used by the altitude estimator.
float verticalAccel() {
  const float sr = sinf(angleRoll * DEG2RAD),  cr = cosf(angleRoll * DEG2RAD);
  const float sp = sinf(anglePitch * DEG2RAD), cp = cosf(anglePitch * DEG2RAD);
  // "up" direction seen in the drone frame = (sin pitch, sin roll cos pitch, cos roll cos pitch)
  const float fUp = accF * sp + accL * sr * cp + accU * cr * cp;   // in g
  return (fUp - 1.0f) * GRAVITY;
}

float tiltCos() {   // cos(roll) * cos(pitch): 1 when level
```

```
  return cosf(angleRoll * DEG2RAD) * cosf(anglePitch * DEG2RAD);
}
```

D8. 4_flight_controller / baro_bmp280.h

```
#pragma once
// BMP280 barometer driver (no library needed). Address 0x76 or 0x77, found automatically.
// Compensation formulas: Bosch BMP280 datasheet, section 3.11.3 (64-bit integer version).
//
// Settings: pressure x8, temperature x1, IIR filter x4, normal mode, standby 0.5 ms
//   -> a new sample about every 23 ms (~44 Hz), noise around 5-10 cm.
// TIP: cover the sensor with a piece of open-cell foam: prop wash and light
//      both disturb the reading.

uint8_t bmpAddr = 0;            // 0 = not found
bool baroOk = false;
float baroPressurePa = 0.0f;    // latest pressure, Pa
float baroTempC = 0.0f;
uint8_t baroFails = 0;

static uint16_t digT1, digP1;
static int16_t  digT2, digT3, digP2, digP3, digP4, digP5, digP6, digP7, digP8, digP9;

bool baroRead() {
  if (!bmpAddr) return false;
  uint8_t b[6];
  if (!i2cReadBytes(bmpAddr, 0xF7, b, 6)) return false;
  const int32_t adcP = ((int32_t)b[0] << 12) | ((int32_t)b[1] << 4) | (b[2] >> 4);
  const int32_t adcT = ((int32_t)b[3] << 12) | ((int32_t)b[4] << 4) | (b[5] >> 4);
  if (adcP == 0x80000 || adcT == 0x80000) return false;     // no measurement yet

  // temperature -> tFine
  int32_t v1 = ((((adcT >> 3) - ((int32_t)digT1 << 1))) * ((int32_t)digT2)) >> 11;
  int32_t v2 = (((((adcT >> 4) - ((int32_t)digT1)) * ((adcT >> 4) - ((int32_t)digT1))) >> 12) *
                ((int32_t)digT3)) >> 14;
  const int32_t tFine = v1 + v2;
  baroTempC = ((tFine * 5 + 128) >> 8) / 100.0f;

  // pressure (64-bit), result in Pa * 256
  int64_t p1 = (int64_t)tFine - 128000;
  int64_t p2 = p1 * p1 * (int64_t)digP6;
  p2 = p2 + ((p1 * (int64_t)digP5) << 17);
  p2 = p2 + (((int64_t)digP4) << 35);
  p1 = ((p1 * p1 * (int64_t)digP3) >> 8) + ((p1 * (int64_t)digP2) << 12);
  p1 = (((((int64_t)1) << 47) + p1)) * ((int64_t)digP1) >> 33;
  if (p1 == 0) return false;
  int64_t p = 1048576 - adcP;
  p = (((p << 31) - p2) * 3125) / p1;
  p1 = (((int64_t)digP9) * (p >> 13) * (p >> 13)) >> 25;
  p2 = (((int64_t)digP8) * p) >> 19;
  p = ((p + p1 + p2) >> 8) + (((int64_t)digP7) << 4);

  const float pa = (float)p / 256.0f;
  if (pa < 30000.0f || pa > 110000.0f) return false;          // nonsense -> ignore
  baroPressurePa = pa;
  return true;
}

bool baroInit() {
  bmpAddr = 0;
  const uint8_t addrs[2] = {0x76, 0x77};
  for (uint8_t i = 0; i < 2; i++) {
    const int id = i2cReadReg(addrs[i], 0xD0);
    if (id == 0x58 || id == 0x60) { bmpAddr = addrs[i]; break; }   // BMP280 / BME280
  }
  if (!bmpAddr) return false;

  i2cWriteReg(bmpAddr, 0xE0, 0xB6);    // soft reset -> sleep mode
```

```
  delay(10);
  uint8_t c[24];
  if (!i2cReadBytes(bmpAddr, 0x88, c, 24)) { bmpAddr = 0; return false; }
  digT1 = (uint16_t)(c[0] | (c[1] << 8));
  digT2 = (int16_t)(c[2] | (c[3] << 8));
  digT3 = (int16_t)(c[4] | (c[5] << 8));
  digP1 = (uint16_t)(c[6] | (c[7] << 8));
  digP2 = (int16_t)(c[8] | (c[9] << 8));
  digP3 = (int16_t)(c[10] | (c[11] << 8));
  digP4 = (int16_t)(c[12] | (c[13] << 8));
  digP5 = (int16_t)(c[14] | (c[15] << 8));
  digP6 = (int16_t)(c[16] | (c[17] << 8));
  digP7 = (int16_t)(c[18] | (c[19] << 8));
  digP8 = (int16_t)(c[20] | (c[21] << 8));
  digP9 = (int16_t)(c[22] | (c[23] << 8));

  i2cWriteReg(bmpAddr, 0xF5, 0x08);    // config: standby 0.5 ms, IIR filter x4 (write in sleep mode)
  i2cWriteReg(bmpAddr, 0xF4, 0x33);    // ctrl_meas: temp x1, pressure x8, normal mode
  delay(60);
  for (uint8_t i = 0; i < 10; i++) {   // wait for the first real sample
    if (baroRead()) return true;
    delay(25);
  }
  bmpAddr = 0;
  return false;
}

// Called at 50 Hz from the main loop. Returns true when a sample was read.
bool baroUpdate() {
  if (!baroOk) return false;
  if (baroRead()) { baroFails = 0; return true; }
  if (++baroFails > 25) baroOk = false;   // 0.5 s without data = sensor lost
  return false;
}
```

D9. 4_flight_controller / altitude.h

```
#pragma once
// Altitude estimation and altitude control.
//
// ESTIMATOR: third-order complementary filter (same idea as ArduPilot's old
// AP_InertialNav). The accelerometer gives fast, smooth motion but drifts; the
// barometer is slow and noisy but never drifts. The filter keeps the best of both
// and also learns the accelerometer's bias:
//     e     = baroAlt - altEst
//     bias += K3 * e * dt
//     vel  += (accV + bias + K2 * e) * dt
//     alt  += (vel + K1 * e) * dt
// with K1 = 3/tau, K2 = 3/tau^2, K3 = 1/tau^3  (all three poles at -1/tau).
//
// CONTROLLER (ALT HOLD and LAND): two loops in a row
//     altitude error  --(ALT_P)-->  wanted vertical speed
//     speed error     --(VEL_P, VEL_I)-->  throttle around the hover throttle

constexpr float ALT_TAU = 1.5f;      // s: bigger = trust the barometer less
constexpr float ALT_K1 = 3.0f / ALT_TAU;
constexpr float ALT_K2 = 3.0f / (ALT_TAU * ALT_TAU);
constexpr float ALT_K3 = 1.0f / (ALT_TAU * ALT_TAU * ALT_TAU);

float groundPressure = 0.0f;   // Pa at the take-off point (set when arming)
float baroAlt = 0.0f;          // m, raw barometer altitude above the take-off point
float altEst = 0.0f;           // m, fused altitude
float velEst = 0.0f;           // m/s, fused vertical speed (up = +)
float accBias = 0.0f;          // m/s^2, learned accelerometer error
float accVf = 0.0f;            // m/s^2, smoothed vertical acceleration
bool altValid = false;

float altTarget = 0.0f;        // m, ALT HOLD target
```

```
float velI = 0.0f;             // us, integral of the speed loop
float hoverThr = HOVER_THROTTLE_GUESS;          // us, base throttle of the vertical controller
float hoverThrLearned = HOVER_THROTTLE_GUESS;   // us, learned while flying level and steady
bool hoverLearnedValid = false;
float hoverLearnTime = 0.0f;

float pressureToAlt(float p) {
  if (groundPressure <= 0.0f) return 0.0f;
  return 44330.0f * (1.0f - powf(p / groundPressure, 0.190295f));
}

// Make the current position "0 m" (called when arming, and by the 'z' command)
void altZero() {
  if (baroPressurePa > 0.0f) groundPressure = baroPressurePa;
  baroAlt = 0.0f;
  altEst = 0.0f;
  velEst = 0.0f;
  altTarget = 0.0f;
}

void altUpdate(float dt, bool newBaroSample) {
  const float a = verticalAccel();
  accVf += (dt / (0.0318f + dt)) * (a - accVf);            // ~5 Hz low-pass against vibration
  if (!baroOk || groundPressure <= 0.0f) { altValid = false; velEst = 0.0f; return; }
  if (newBaroSample) baroAlt = pressureToAlt(baroPressurePa);
  const float e = baroAlt - altEst;
  accBias = clampf(accBias + ALT_K3 * e * dt, -2.0f, 2.0f);
  velEst += (accVf + accBias + ALT_K2 * e) * dt;
  altEst += (velEst + ALT_K1 * e) * dt;
  altValid = true;
}

// Start a vertical-control session (ALT HOLD or LAND) from a known hover throttle
void altCtrlReset(float hover) {
  hoverThr = clampf(hover, 1250.0f, 1800.0f);
  velI = 0.0f;
  altTarget = altEst;
}

// Speed loop: returns the throttle in us
float altCtrlUpdate(float velTarget, float dt) {
  const float velErr = velTarget - velEst;
  const float newI = clampf(velI + VEL_I * velErr * dt, -VEL_I_MAX, VEL_I_MAX);
  float thr = hoverThr + VEL_P * velErr + newI;
  const float tc = fmaxf(tiltCos(), 0.7f);                    // tilted = less lift: add some
  thr = 1000.0f + (thr - 1000.0f) / tc;
  const float lo = (float)MOTOR_IDLE + 30.0f, hi = (float)THROTTLE_MAX;
  // anti wind-up: only keep the new integral if the output is not saturated the same way
  if (!((thr >= hi && velErr > 0) || (thr <= lo && velErr < 0))) velI = newI;
  return clampf(thr, lo, hi);
}

// ALT HOLD: throttle stick near the middle = hold; above/below = climb/descend
float altHoldVelTarget() {
  const int off = (int)rc[CH_THR] - 1500;
  if (abs(off) > ALT_STICK_DEADBAND) {
    const float s = clampf((abs(off) - ALT_STICK_DEADBAND) / (float)(500 - ALT_STICK_DEADBAND), 0.0f, 1.0f);
    altTarget = altEst;                                       // follow while the pilot moves
    return off > 0 ? s * ALT_MAX_CLIMB : -s * ALT_MAX_DESCENT;
  }
  return clampf(ALT_P * (altTarget - altEst), -ALT_MAX_DESCENT, ALT_MAX_CLIMB);
}

float landVelTarget() {
  return -(altEst > LAND_SLOW_ALT ? LAND_SPEED_FAST : LAND_SPEED_SLOW);
}
```

D10. 4_flight_controller / battery.h

```
#pragma once
// Battery monitoring: voltage (H5 divider on GPIO 1), optional current sensor,
// percentage, and the LOW / CRITICAL states that trigger warnings and auto-landing.
//
// Voltage divider:  Vpin = Vbat * R4 / (R3 + R4)   ->   Vbat = Vpin * (R3 + R4) / R4
// With R3 = 10k and R4 = 3.3k, a full 3S (12.6 V) gives 3.13 V on the pin.
//
// SAG COMPENSATION (no current sensor needed)
// Under load a LiPo's voltage drops because of its internal resistance, so the raw
// voltage in flight says "40 %" for a battery that is really at 60 %. The drop is
// about proportional to the battery current, and the current is about proportional
// to the motor load  L = sum(u^3)  (u = motor command 0..1; motor power grows with
// speed cubed). Just before arming the battery is resting, so in the first seconds
// of flight we learn  K = (V_rest - V_now) / L  and then estimate the resting voltage
// as  V_rest = V_now + K * L.  Percentage and LOW / CRITICAL use this estimate.
// A hard floor on the raw voltage (CELL_ABS_MIN_V) still protects the battery.

enum BattState : uint8_t { BATT_OK = 0, BATT_LOW = 1, BATT_CRITICAL = 2 };

float vbat = 0.0f;          // V, measured (filtered), sags under load
float vRest = 0.0f;         // V, estimated resting voltage (sag compensated)
float vCell = 0.0f;         // V per cell, from vRest
float vCellRaw = 0.0f;      // V per cell, measured
float battAmps = 0.0f;      // A (0 without a current sensor)
float mAhUsed = 0.0f;
int battPct = -1;           // 0..100 from vRest, -1 = unknown
bool battPresent = false;   // false when only USB powers the board
BattState battState = BATT_OK;
float lowTimer = 0.0f, critTimer = 0.0f, okTimer = 0.0f;

float loadF = 0.0f;         // motor load, filtered like vbat
float sagK = 0.0f;          // V per unit of load (learned in flight)
float sagLearnT = 0.0f;     // s of learning in this flight
float vRest0 = 0.0f;        // V, resting voltage just before arming

// Resting voltage per cell -> charge (%). Typical LiPo curve.
static const float PCT_V[] = {3.30f, 3.50f, 3.61f, 3.69f, 3.71f, 3.73f, 3.75f, 3.77f, 3.79f, 3.80f, 3.82f,
                              3.84f, 3.85f, 3.87f, 3.91f, 3.95f, 3.98f, 4.02f, 4.08f, 4.11f, 4.15f, 4.20f};
static const uint8_t PCT_P[] = {0, 2, 5, 10, 15, 20, 25, 30, 35, 40, 45,
                                50, 55, 60, 65, 70, 75, 80, 85, 90, 95, 100};

int lipoPercent(float cellV) {
  const int n = sizeof(PCT_V) / sizeof(PCT_V[0]);
  if (cellV <= PCT_V[0]) return 0;
  if (cellV >= PCT_V[n - 1]) return 100;
  for (int i = 1; i < n; i++) {
    if (cellV < PCT_V[i]) {
      const float f = (cellV - PCT_V[i - 1]) / (PCT_V[i] - PCT_V[i - 1]);
      return (int)(PCT_P[i - 1] + f * (PCT_P[i] - PCT_P[i - 1]) + 0.5f);
    }
  }
  return 100;
}

float motorLoad() {          // ~ battery current, in "motor units" (0..4)
  float s = 0.0f;
  for (uint8_t m = 0; m < 4; m++) {
    const float u = clampf((motorOut[m] - 1050.0f) / 950.0f, 0.0f, 1.0f);
    s += u * u * u;
  }
  return s;
}

// Called 10 times per second. learnSag = armed (the battery was resting just before).
void battUpdate(float dt, bool learnSag) {
  const float v = analogReadMilliVolts(VBAT_PIN) / 1000.0f * VBAT_DIVIDER * VBAT_CAL;
  vbat = (vbat < 1.0f) ? v : vbat + 0.2f * (v - vbat);
  loadF += 0.2f * (motorLoad() - loadF);              // same filter, so both lag together
```

```
  battPresent = vbat > 2.5f * BATTERY_CELLS;

  if (CURRENT_PIN >= 0) {
    const float a = (analogReadMilliVolts(CURRENT_PIN) / 1000.0f - CURRENT_OFFSET_V) * CURRENT_A_PER_V;
    battAmps += 0.3f * (a - battAmps);
    if (battAmps > 0.0f) mAhUsed += battAmps * 1000.0f * dt / 3600.0f;
  }
  if (!battPresent) { vRest = vCell = vCellRaw = 0.0f; battPct = -1; lowTimer = critTimer = okTimer = 0.0f;
return; }

  // learn the sag during the first 10 s with the motors working (take-off, hover)
  if (learnSag && loadF > 0.25f && vRest0 > 0.0f && sagLearnT < 10.0f) {
    const float k = clampf((vRest0 - vbat) / loadF, 0.0f, 3.0f);
    sagK = (sagLearnT == 0.0f) ? k : sagK + 0.1f * (k - sagK);
    sagLearnT += dt;
  }
  // during the first 2 s of learning K is not ready yet: the battery cannot lose
  // much charge in 2 s, so keep the resting voltage measured before arming
  vRest = (learnSag && sagLearnT > 0.0f && sagLearnT < 2.0f) ? vRest0 : vbat + sagK * loadF;
  vCell = vRest / BATTERY_CELLS;
  vCellRaw = vbat / BATTERY_CELLS;
  battPct = lipoPercent(vCell);

  // A state must last 5 s (short dips on punch-outs are ignored). In flight it stays latched.
  // On the ground (disarmed) a healthy battery for 3 s clears it (fresh pack on the bench).
  lowTimer  = (vCell < CELL_WARN_V) ? lowTimer + dt : 0.0f;
  critTimer = (vCell < CELL_CRITICAL_V || vCellRaw < CELL_ABS_MIN_V) ? critTimer + dt : 0.0f;
  okTimer   = (!learnSag && vCell >= CELL_MIN_ARM_V) ? okTimer + dt : 0.0f;
  if (critTimer >= 5.0f) battState = BATT_CRITICAL;
  else if (lowTimer >= 5.0f && battState == BATT_OK) battState = BATT_LOW;
  else if (okTimer >= 3.0f) battState = BATT_OK;
}

void battResetFlight() {   // at arming (motors still off: the battery is resting)
  battState = BATT_OK;
  lowTimer = critTimer = okTimer = 0.0f;
  mAhUsed = 0.0f;
  vRest0 = vbat;
  sagLearnT = 0.0f;        // sagK keeps the last value until it is learned again
}
```

D11. 4_flight_controller / status_beeper.h

```
#pragma once
// Buzzer + RGB LED. Everything is non-blocking: beepUpdate() runs every loop
// (called from statusUpdate() in the main sketch, which also picks the LED colour).
//
// Buzzer (one-shots)                    Buzzer (repeating, most important wins)
//   ready to arm / log saved .. 1 long    landing ............ beep-beep (0.25 s)
//   armed, mode accepted ...... 1 short   battery CRITICAL ... fast chirps
//   disarmed .................. 2 short   battery LOW ........ short beep every second
//   refused (arm or mode) ..... 3 fast    lost drone ......... chirp every 2 s (after a crash /
//                                                              radio-loss landing, until re-armed)
//
// RGB LED: yellow = starting, green = ready (blinking = not ready), blue = armed MANUAL,
//          purple = ALT HOLD, orange blinking = LAND, yellow flashes = battery LOW,
//          red blinking = battery CRITICAL or MPU6050 missing, white = saving the log

enum BeepRepeat : uint8_t { REP_NONE = 0, REP_BATT_LOW, REP_BATT_CRIT, REP_LANDING, REP_LOST };
struct RepDef { uint16_t on, off; };
static const RepDef REP_DEFS[] = {{0, 0}, {80, 920}, {80, 220}, {250, 250}, {120, 1880}};

BeepRepeat beepRepeat = REP_NONE;
struct Shot { uint8_t remaining; uint16_t on, off; bool phaseOn; uint32_t t0; };
Shot shot = {0, 0, 0, false, 0};
bool buzzerIsOn = false;

void buzzerWrite(bool on) {
```

```
  if (BUZZER_PIN < 0 || on == buzzerIsOn) return;
  buzzerIsOn = on;
  if (BUZZER_PASSIVE) {
#if ESP_ARDUINO_VERSION_MAJOR >= 3
    ledcWrite(BUZZER_PIN, on ? 128 : 0);
#else
    ledcWrite(4, on ? 128 : 0);
#endif
  } else {
    digitalWrite(BUZZER_PIN, (on == BUZZER_ACTIVE_HIGH) ? HIGH : LOW);
  }
}

// Call AFTER escInit() (the motors must own LEDC channels 0-3)
void statusBegin() {
  if (BUZZER_PIN >= 0) {
    if (BUZZER_PASSIVE) {
#if ESP_ARDUINO_VERSION_MAJOR >= 3
      ledcAttach(BUZZER_PIN, 2700, 8);
#else
      ledcSetup(4, 2700, 8);
      ledcAttachPin(BUZZER_PIN, 4);
#endif
    } else {
      pinMode(BUZZER_PIN, OUTPUT);
    }
    buzzerIsOn = true;
    buzzerWrite(false);
  }
}

void beep(uint8_t n, uint16_t onMs = 80, uint16_t offMs = 120) {
  shot = {n, onMs, offMs, true, (uint32_t)millis()};
  buzzerWrite(true);
}

void beepUpdate(uint32_t now) {
  if (shot.remaining) {                       // one-shot beeps win over the repeating pattern
    if (shot.phaseOn && now - shot.t0 >= shot.on) {
      buzzerWrite(false); shot.phaseOn = false; shot.t0 = now; shot.remaining--;
    } else if (!shot.phaseOn && now - shot.t0 >= shot.off) {
      buzzerWrite(true); shot.phaseOn = true; shot.t0 = now;
    }
    return;
  }
  const RepDef &r = REP_DEFS[beepRepeat];
  if (r.on == 0) { buzzerWrite(false); return; }
  buzzerWrite((now % (r.on + r.off)) < r.on);
}

// ---------------- RGB LED ----------------
uint32_t ledColor = 0xFFFFFFFF;

void ledWrite(uint8_t r, uint8_t g, uint8_t b) {
  if (RGB_LED_PIN < 0) return;
  const uint32_t c = ((uint32_t)r << 16) | ((uint32_t)g << 8) | b;
  if (c == ledColor) return;                  // only talk to the LED when the colour changes
  ledColor = c;
#if ESP_ARDUINO_VERSION_MAJOR >= 3
  rgbLedWrite(RGB_LED_PIN, r, g, b);
#else
  neopixelWrite(RGB_LED_PIN, r, g, b);
#endif
}
```

D12. 4_flight_controller / flight_log.h

```
#pragma once
// Flight log.
//  - During flight: one record (44 bytes) every 20 ms goes into PSRAM (RAM only:
//    writing to flash in flight would pause the CPU and disturb the control loop).
//  - After disarming: the records are saved to flash as /log_NNN.bin (FFat partition,
//    "16M Flash (3MB APP/9.9MB FATFS)"). Wait for the 3 beeps before unplugging.
//  - On the bench (USB, H1 unplugged): type 'l' to list, 'd' to dump the newest log
//    as CSV, 'd7' for log 7, 'X' to erase all. tools/flight_log.py does it for you.
#include "FFat.h"

struct __attribute__((packed)) LogRec {
  uint32_t t_ms;
  int16_t roll_cd, pitch_cd;        // 0.01 deg
  int16_t rollSp_cd, pitchSp_cd;    // 0.01 deg (wanted angles)
  int16_t yawRate_dd;               // 0.1 deg/s
  int16_t alt_cm, altTgt_cm, baroAlt_cm;
  int16_t vz_cms;                   // cm/s
  uint16_t thrIn, thrOut;           // us
  uint16_t m[4];                    // us
  uint16_t vbat_mv;                 // measured (sags under load)
  uint16_t vrest_mv;                // estimated resting voltage (sag compensated)
  int16_t amps_ca;                  // 0.01 A (current sensor, else 0)
  uint8_t battPct;                  // % from vrest
  uint8_t mode;                     // 0 MANUAL, 1 ALT HOLD, 2 LAND
  uint8_t flags;                    // see LOGF_*
  uint8_t spare;
};
static_assert(sizeof(LogRec) == 44, "LogRec must stay 44 bytes");
constexpr uint16_t LOG_VERSION = 2;

enum : uint8_t { LOGF_ARMED = 1, LOGF_AIRBORNE = 2, LOGF_RADIO_OK = 4, LOGF_BATT_LOW = 8,
                 LOGF_BATT_CRIT = 16, LOGF_BARO_OK = 32, LOGF_LAND_RADIO = 64, LOGF_LAND_BATT = 128 };

struct __attribute__((packed)) LogHeader {
  char magic[4];        // "DLOG"
  uint16_t version;     // LOG_VERSION
  uint16_t recSize;     // sizeof(LogRec)
  uint32_t count;
  uint16_t logHz;
  uint16_t reserved;
};

LogRec *logBuf = nullptr;
uint32_t logCap = 0, logCount = 0;
bool logActive = false;
bool fsOk = false;
bool logPendingSave = false;
uint32_t logStartMs = 0;

void logBegin() {
  if (!LOG_ENABLED) return;
  if (psramFound()) {
    logCap = (2UL * 1024 * 1024) / sizeof(LogRec);            // 2 MB = ~16 min at 50 Hz
    logBuf = (LogRec *)ps_malloc(logCap * sizeof(LogRec));
  }
  if (!logBuf) {
    logCap = (48UL * 1024) / sizeof(LogRec);                  // no PSRAM: ~22 s
    logBuf = (LogRec *)malloc(logCap * sizeof(LogRec));
  }
  if (!logBuf) logCap = 0;
  fsOk = FFat.begin(true);
}

void logStart() { logCount = 0; logStartMs = millis(); logActive = (logBuf != nullptr); logPendingSave =
false; }
void logStop()  { if (logActive && logCount > 0) logPendingSave = fsOk; logActive = false; }
void logAdd(const LogRec &r) { if (logActive && logCount < logCap) logBuf[logCount++] = r; }
```

```
static int logIndexFromName(const char *name) {
  const char *p = strstr(name, "log_");
  return p ? atoi(p + 4) : -1;
}

// newest / oldest index and file count
static void logScan(int &newest, int &oldest, int &count) {
  newest = -1; oldest = 100000; count = 0;
  File root = FFat.open("/");
  if (!root) return;
  File f = root.openNextFile();
  while (f) {
    const int idx = logIndexFromName(f.name());
    if (idx >= 0) { count++; if (idx > newest) newest = idx; if (idx < oldest) oldest = idx; }
    f = root.openNextFile();
  }
}

static void logPath(int idx, char *out) { snprintf(out, 20, "/log_%03d.bin", idx); }

// Saves the RAM log to flash. Blocking (a few seconds) -> only call while DISARMED.
bool logSave() {
  logPendingSave = false;
  if (!fsOk || logCount == 0) return false;
  int newest, oldest, count;
  logScan(newest, oldest, count);
  const size_t need = sizeof(LogHeader) + logCount * sizeof(LogRec) + 8192;
  for (int guard = 0; guard < 100 && count > 0 &&
                      (count >= LOG_MAX_FILES || FFat.freeBytes() < need); guard++) {   // make room
    char p[20]; logPath(oldest, p); FFat.remove(p);
    logScan(newest, oldest, count);
  }
  const int idx = (newest < 0 ? 1 : newest + 1);
  char path[20]; logPath(idx, path);
  File f = FFat.open(path, FILE_WRITE);
  if (!f) return false;
  LogHeader h = {{'D', 'L', 'O', 'G'}, LOG_VERSION, (uint16_t)sizeof(LogRec), logCount, LOG_HZ, 0};
  f.write((const uint8_t *)&h, sizeof(h));
  const uint8_t *src = (const uint8_t *)logBuf;
  size_t left = logCount * sizeof(LogRec);
  while (left) {
    const size_t n = left > 8192 ? 8192 : left;
    if (f.write(src, n) != n) { f.close(); return false; }
    src += n; left -= n;
  }
  f.close();
  if (DEBUG) Serial.printf("Log saved: %s (%lu records, %.1f s)\n", path,
                           (unsigned long)logCount, logCount / (float)LOG_HZ);
  return true;
}

void logList() {
  if (!fsOk) { Serial.println(F("No flash file system (check Partition Scheme: ...FATFS).")); return; }
  Serial.println(F("Saved flight logs:"));
  File root = FFat.open("/");
  File f = root ? root.openNextFile() : File();
  int n = 0;
  while (f) {
    if (logIndexFromName(f.name()) >= 0) {
      Serial.printf("  %-14s %7lu bytes  %6.1f s\n", f.name(), (unsigned long)f.size(),
                    (f.size() - sizeof(LogHeader)) / (float)sizeof(LogRec) / LOG_HZ);
      n++;
    }
    f = root.openNextFile();
  }
  if (!n) Serial.println(F("  (none)"));
  Serial.printf("Flash: %lu KB free of %lu KB\n", (unsigned long)(FFat.freeBytes() / 1024),
                (unsigned long)(FFat.totalBytes() / 1024));
}

static void logPrintCsvHeader() {
  Serial.println(F("t_s,mode,armed,airborne,radio_ok,batt_low,batt_crit,baro_ok,land_radio,land_batt,"
```

```
                  "roll,pitch,roll_sp,pitch_sp,yaw_rate,alt,alt_target,baro_alt,vz,"
                  "thr_in,thr_out,m1,m2,m3,m4,vbat,vrest,batt_pct,current"));
}

static void logPrintCsvRow(const LogRec &r, uint32_t t0) {
  Serial.printf("%.3f,%u,%u,%u,%u,%u,%u,%u,%u,%u,%.2f,%.2f,%.2f,%.2f,%.1f,%.2f,%.2f,%.2f,%.2f,"
                "%u,%u,%u,%u,%u,%u,%.2f,%.2f,%u,%.2f\n",
                (r.t_ms - t0) / 1000.0f, r.mode,
                (r.flags & LOGF_ARMED) ? 1 : 0, (r.flags & LOGF_AIRBORNE) ? 1 : 0,
                (r.flags & LOGF_RADIO_OK) ? 1 : 0, (r.flags & LOGF_BATT_LOW) ? 1 : 0,
                (r.flags & LOGF_BATT_CRIT) ? 1 : 0, (r.flags & LOGF_BARO_OK) ? 1 : 0,
                (r.flags & LOGF_LAND_RADIO) ? 1 : 0, (r.flags & LOGF_LAND_BATT) ? 1 : 0,
                r.roll_cd / 100.0f, r.pitch_cd / 100.0f, r.rollSp_cd / 100.0f, r.pitchSp_cd / 100.0f,
                r.yawRate_dd / 10.0f, r.alt_cm / 100.0f, r.altTgt_cm / 100.0f, r.baroAlt_cm / 100.0f,
                r.vz_cms / 100.0f, r.thrIn, r.thrOut, r.m[0], r.m[1], r.m[2], r.m[3],
                r.vbat_mv / 1000.0f, r.vrest_mv / 1000.0f, r.battPct, r.amps_ca / 100.0f);
}

// Prints a log as CSV between BEGIN_LOG and END_OF_LOG. idx < 0 = newest.
void logDump(int idx) {
  if (!fsOk) {                                  // no flash: dump what is still in RAM
    Serial.printf("BEGIN_LOG ram %lu\n", (unsigned long)logCount);
    logPrintCsvHeader();
    for (uint32_t i = 0; i < logCount; i++) logPrintCsvRow(logBuf[i], logBuf[0].t_ms);
    Serial.println(F("END_OF_LOG"));
    return;
  }
  if (idx < 0) { int oldest, count; logScan(idx, oldest, count); }
  if (idx < 0) { Serial.println(F("No saved log.")); return; }
  char path[20]; logPath(idx, path);
  File f = FFat.open(path, FILE_READ);
  LogHeader h;
  if (!f || f.read((uint8_t *)&h, sizeof(h)) != sizeof(h) || memcmp(h.magic, "DLOG", 4) != 0) {
    Serial.printf("Cannot read %s\n", path);
    return;
  }
  if (h.version != LOG_VERSION || h.recSize != sizeof(LogRec)) {
    Serial.printf("%s was written by another firmware version (v%u). Erase old logs with X.\n", path,
h.version);
    return;
  }
  Serial.printf("BEGIN_LOG %s %lu\n", path, (unsigned long)h.count);
  logPrintCsvHeader();
  LogRec r;
  uint32_t t0 = 0;
  for (uint32_t i = 0; i < h.count; i++) {
    if (f.read((uint8_t *)&r, sizeof(r)) != sizeof(r)) break;
    if (i == 0) t0 = r.t_ms;
    logPrintCsvRow(r, t0);
  }
  f.close();
  Serial.println(F("END_OF_LOG"));
}

void logEraseAll() {
  if (!fsOk) return;
  int newest, oldest, count;
  logScan(newest, oldest, count);
  for (int guard = 0; guard < 1000 && count > 0; guard++) {
    char p[20]; logPath(oldest, p); FFat.remove(p);
    logScan(newest, oldest, count);
  }
  Serial.println(count ? F("Some logs could not be erased.") : F("All logs erased."));
}
```